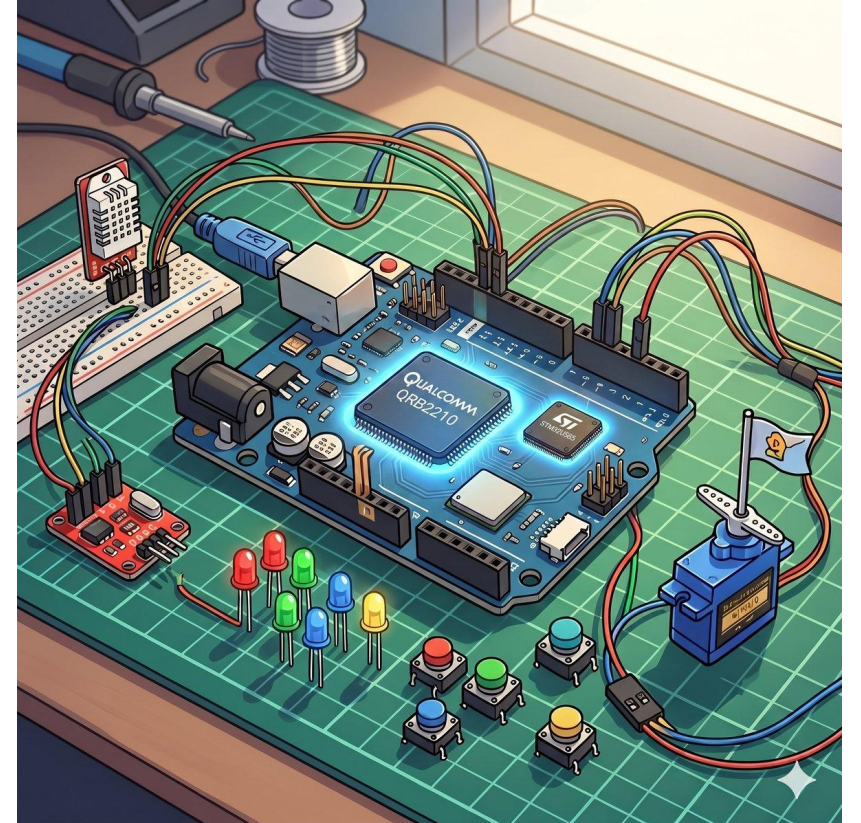


GenAI at the Edge

Introduction



Prof. Marcelo J. Rovai

rovai@unifei.edu.br

UNIFEI - Federal University of Itajuba, Brazil

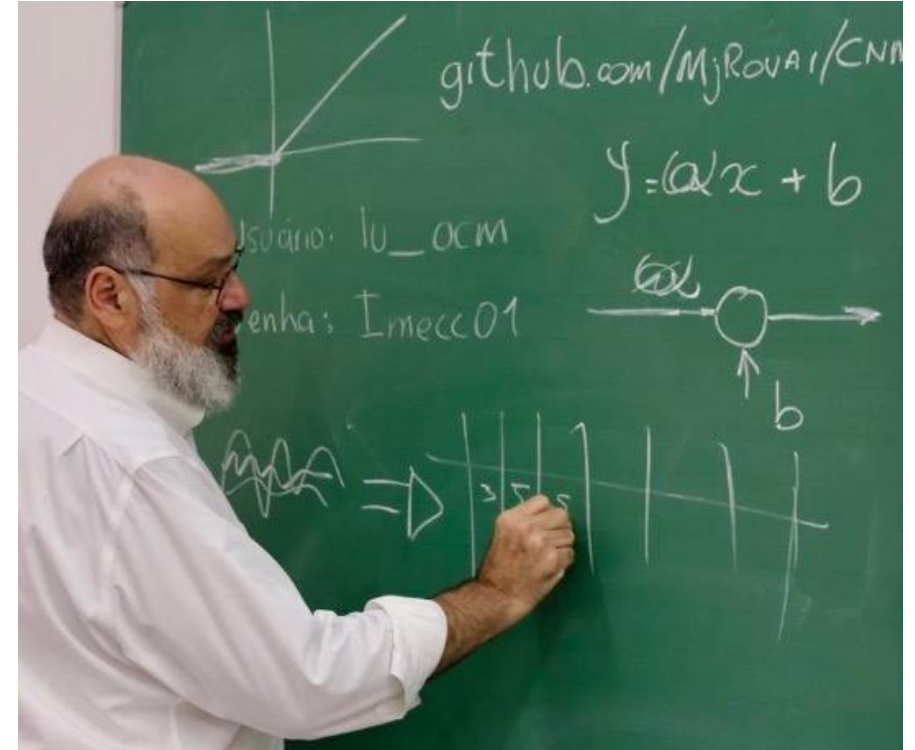
AIEng4D - Academic Network Co-Chair

EdgeAIP - Academia-Industry Partnership Co-Chair

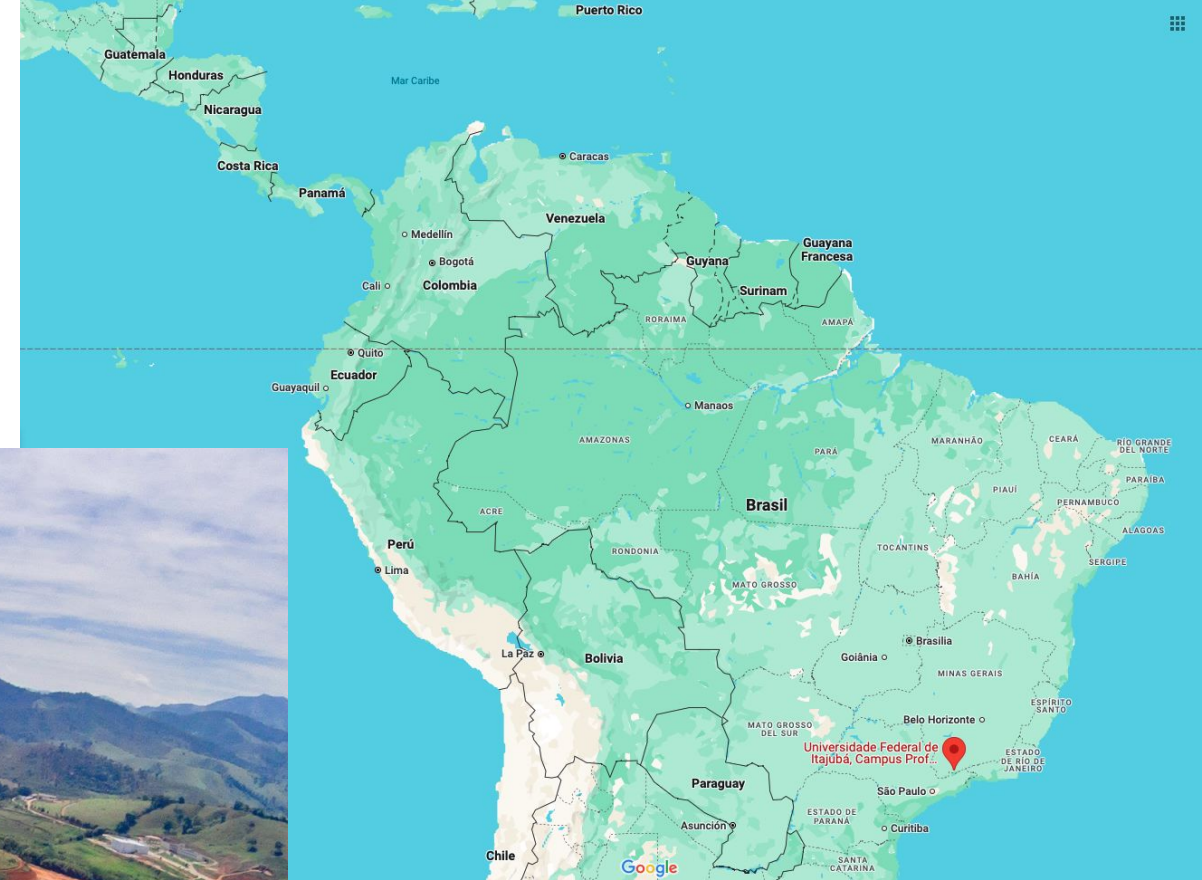


Who I am

- Brazilian from São Paulo, **Data Science Master's degree from UDD**, Chile, and **MBA from IBMEC (INSAPER)**, Brazil.
- Graduated in 1982 as an **Engineer from UNIFEI** with **Specialization from Poli/USP**, both in Brazil.
- Worked as a **teacher, engineer, and executive** in several technology companies such as CDT/ETEP, AVIBRAS Aeroespacial, SID Informática, ATT-GIS, NCR, DELL, COMPAQ (HP), and more recently at IGT as a Regional VP, and a Senior Advisor for Latin America.
- **Write about electronics**, publishing in sites as MJRoBot.org (Editor/Writer), Hackster.io (#1 Contributor), Instructables.com, and Medium.com (TDS – Towards Data Science).
- **Professor** at UNIFEI (IESTI), teaching “Machine Learning applied to Embedded Devices” course (IESTI01) and EdgeAI Engineering (IESTI05).
- **AiEng4D group** and the **EDGE AIP** – the Academia-Industry Partnership of **EDGEAI Foundation** Co-Chair, promoting EdgeAI education in developing countries.



Marcelo Rovai



Itajubá,
Minas Gerais,
Brazil

GenAI: Introduction

Generative AI (GenAI)

Generative AI is an artificial intelligence system capable of creating new, original content across various media — **text, images, audio, and video**. These systems learn patterns from existing data and use that knowledge to **generate novel outputs that didn't previously exist**.

When used for generative tasks, these model families are all types of GenAI:

LLM

**Large
Language
Models**

GPT, Claude,
Gemini, Qwen

SLM

**Small
Language
Models**

Gemma, Phi,
Llama, Qwen

VLM

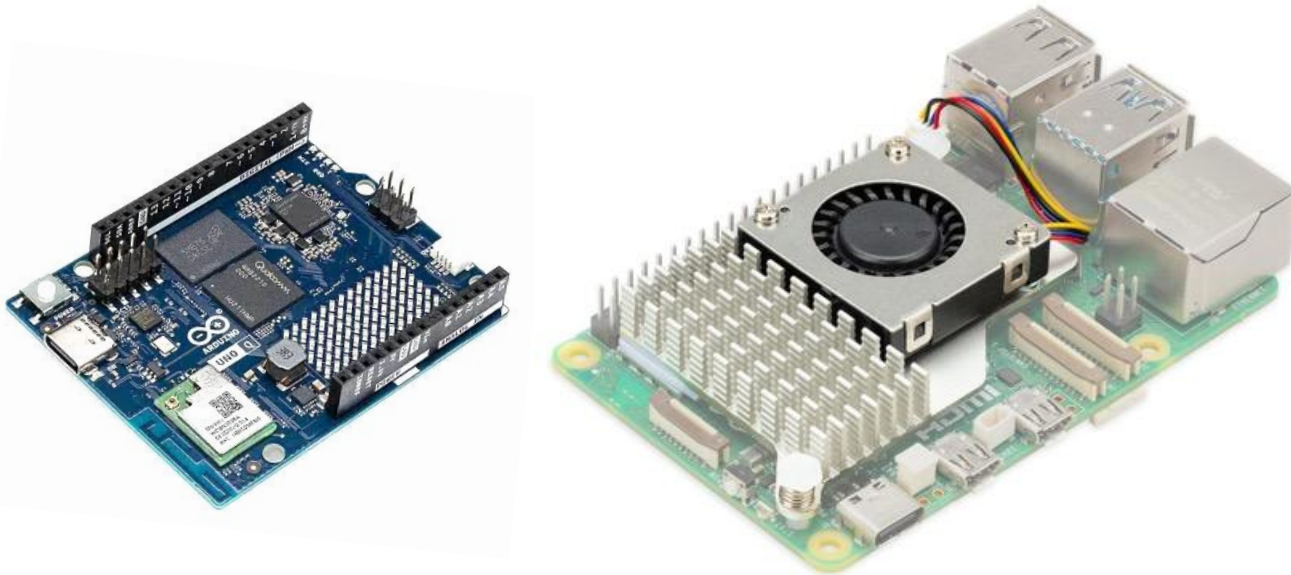
**Visual-
Language
Models**

Gemini, Gemma,
MoonDream, GPT,
PaliGemma,
Florence

GenAI at the Edge

"GenAI at the Edge means the model lives on the device — not in a data center."

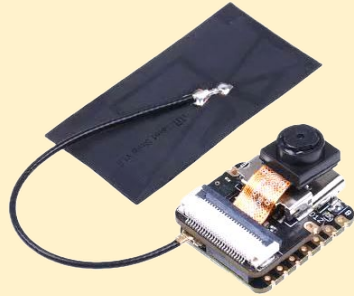
While LLMs typically require cloud infrastructure, compact SLMs* and small VLMs can run directly on edge devices.



* Current trends point to 2026 as the year of SLMs (Models < 7B to 10B), with specialized models surpassing generalist ones in specific tasks, such as programming and medical diagnoses.

Sustainability Matters

0.5W



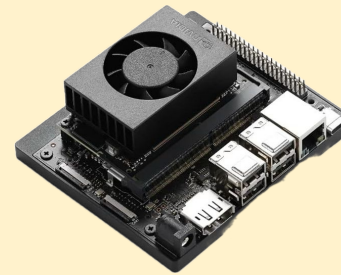
5W-



15W



20W+



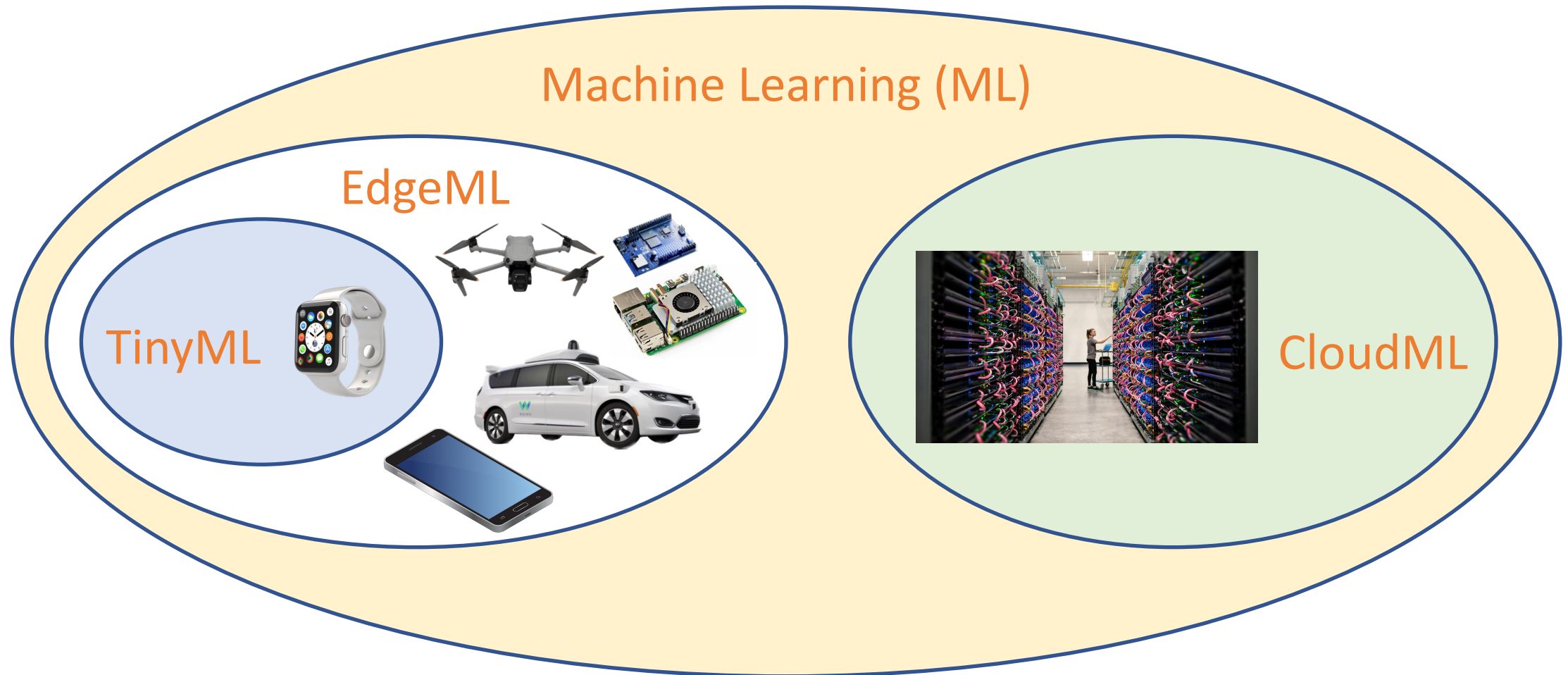
100W+



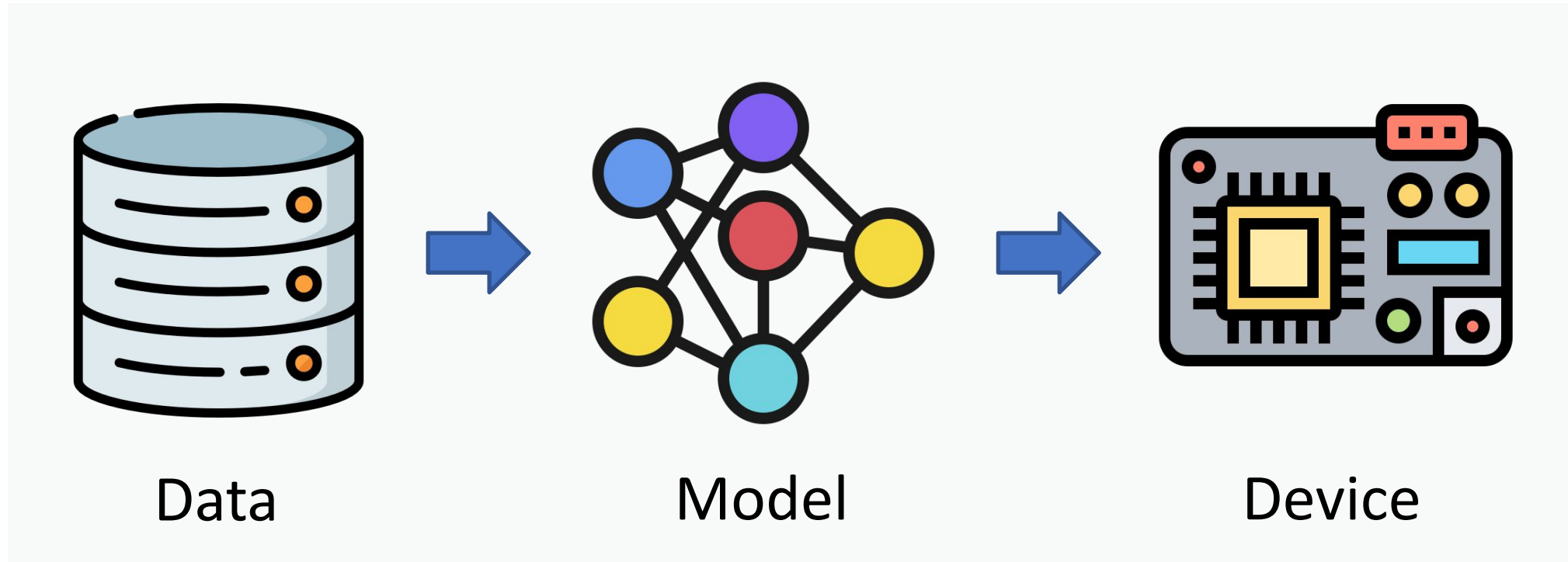
100x more energy efficient than cloud AI

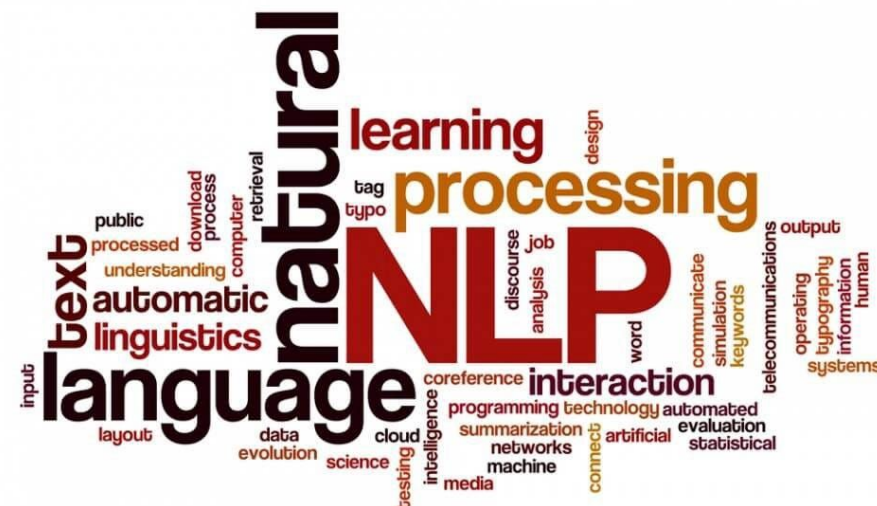
EdgeAI (ML) & TinyML

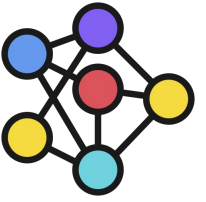
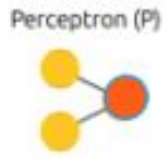
From Tiny to Cloud: One AI, Many Scales



ML Deployment Pipeline







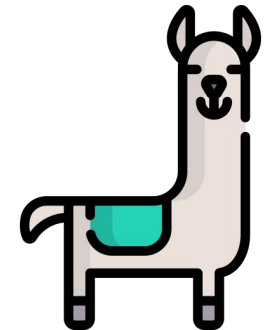
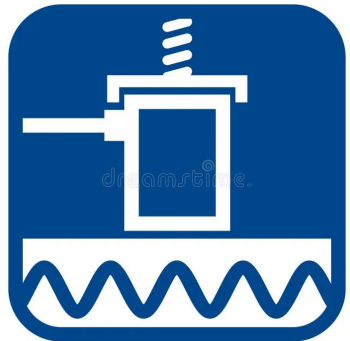
Vibration
Analysis

Image
Classification

Text
Generation

Image
Generation

Large Language
Models- LLMs



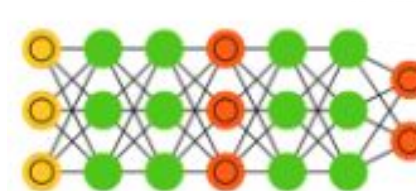
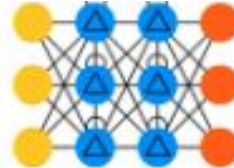
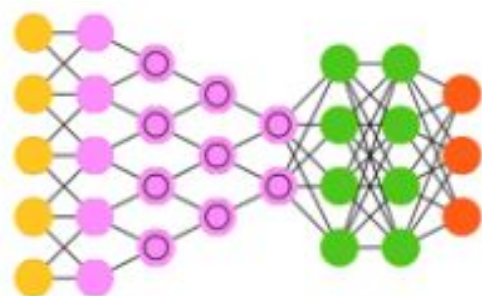
MLP - Deep Neural Network

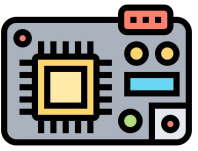
CNN - Convolutional NN

RNN - Recurrent NN (GRU/LSTM)

GAN - Generative Adversarial Net.

TF - Transformer (Attention)





Datasets Preprocessing

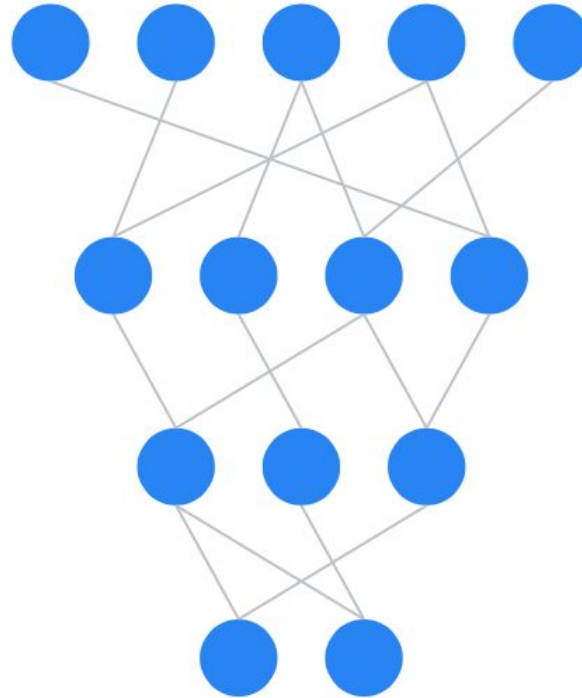
Sound

Vision

Vibration

Text

Quantization Pruning, Distillation

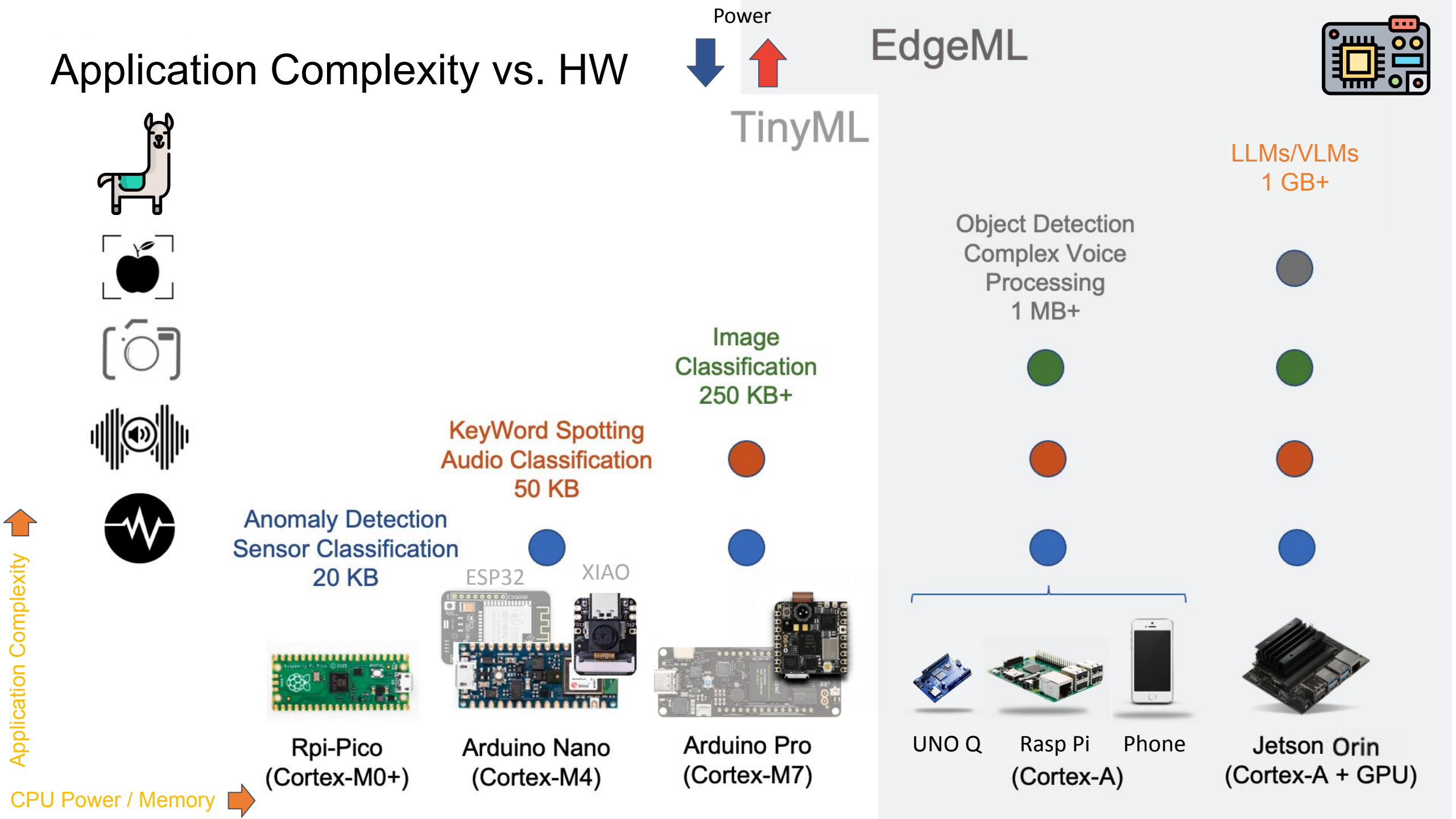


Resource constraints



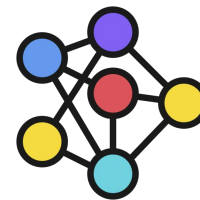
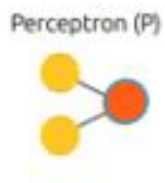
End-to-end **TinyML/EdgeAI** Application

Application Complexity vs. HW



LLM / SLM

Large Language Model / Small Language Models



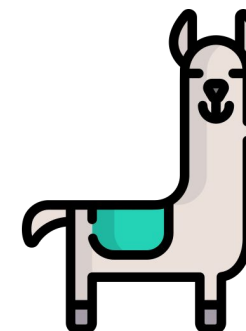
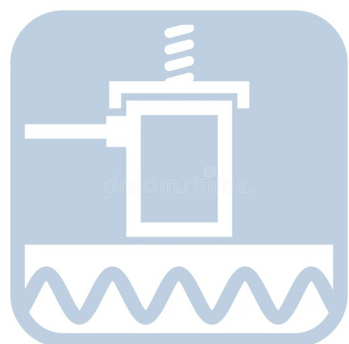
Vibration
Analysis

Image
Classification

Text
Generation

Image
Generation

Large Language
Models- LLMs



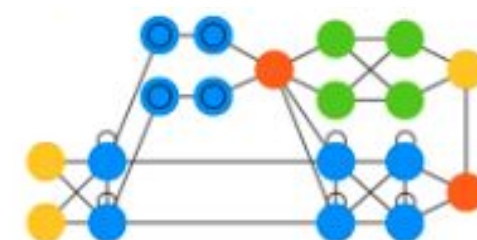
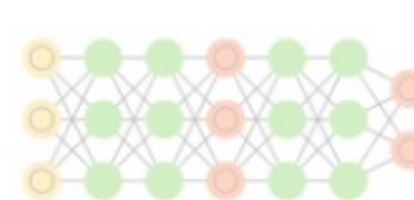
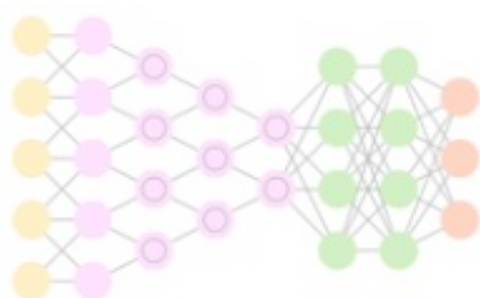
MLP - Deep Neural Network

CNN - Convolutional NN

RNN - Recurrent NN (GRU/LSTM)

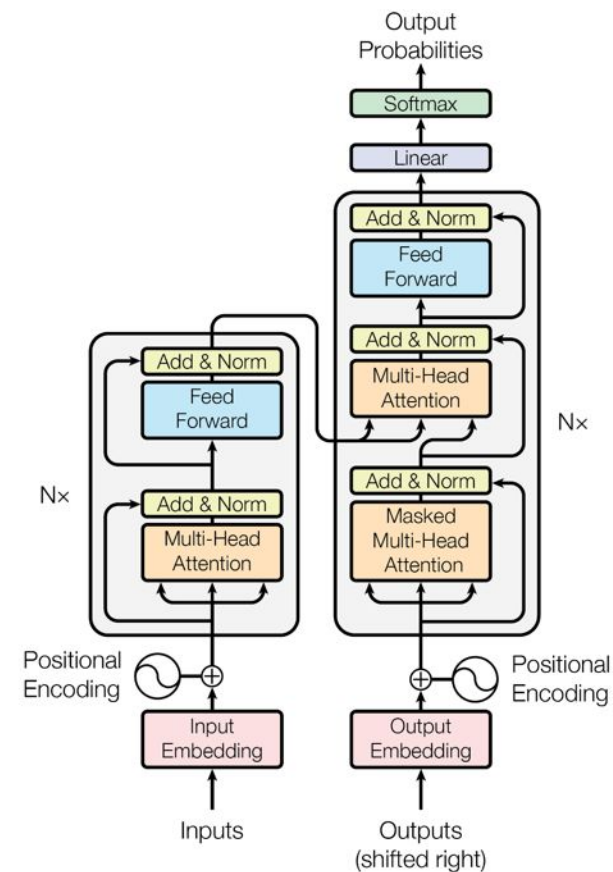
GAN - Generative Adversarial Net.

TF - Transformer (Attention)



LLM/SLM – Large /Small Language Model

Large Language Models (LLMs) and SLMs are advanced neural networks based on the **Transformer architecture** that excel in understanding and generating human language. They represent a significant evolution from earlier sequence-based models like **RNNs**, which surpass them in handling long-range dependencies and parallel processing efficiency.



The Illustrated Transformer

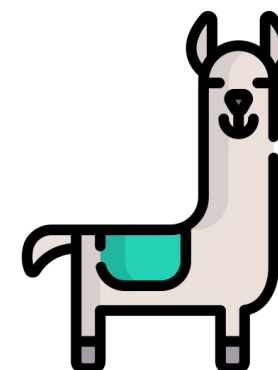
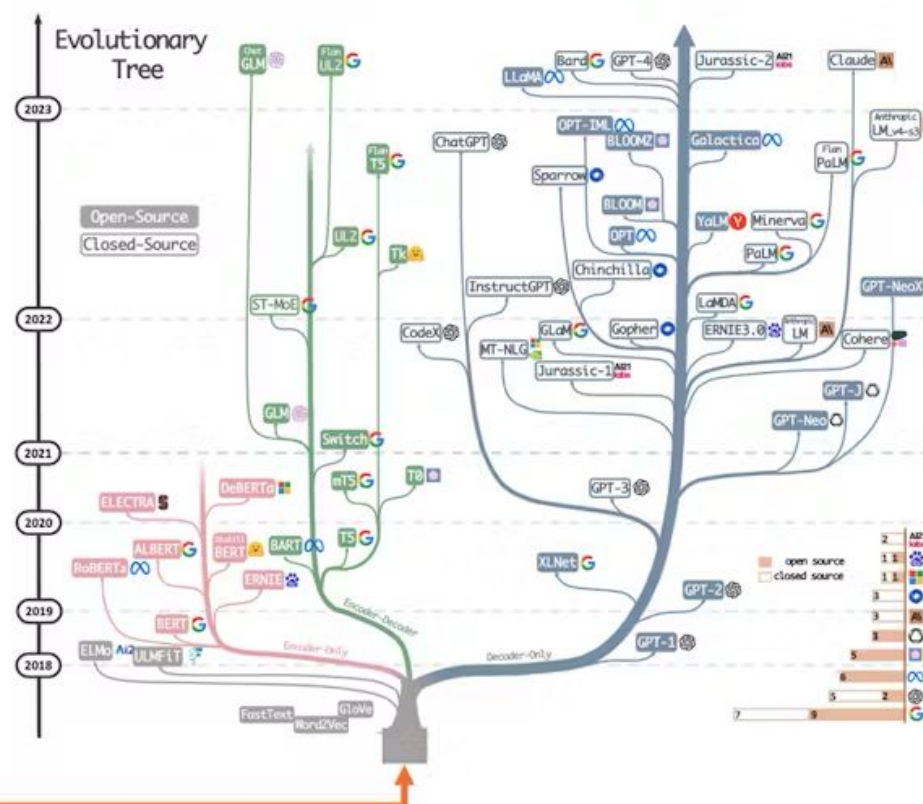
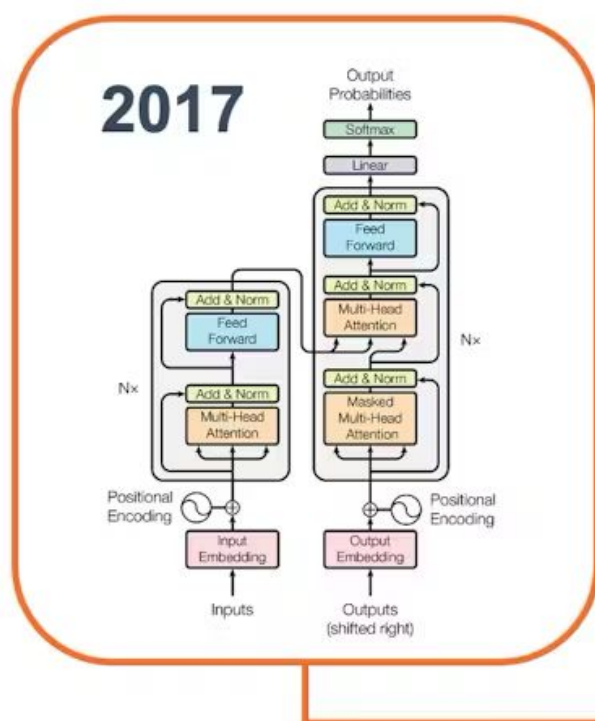
Transformers to LLMs and SLMs

Open - China

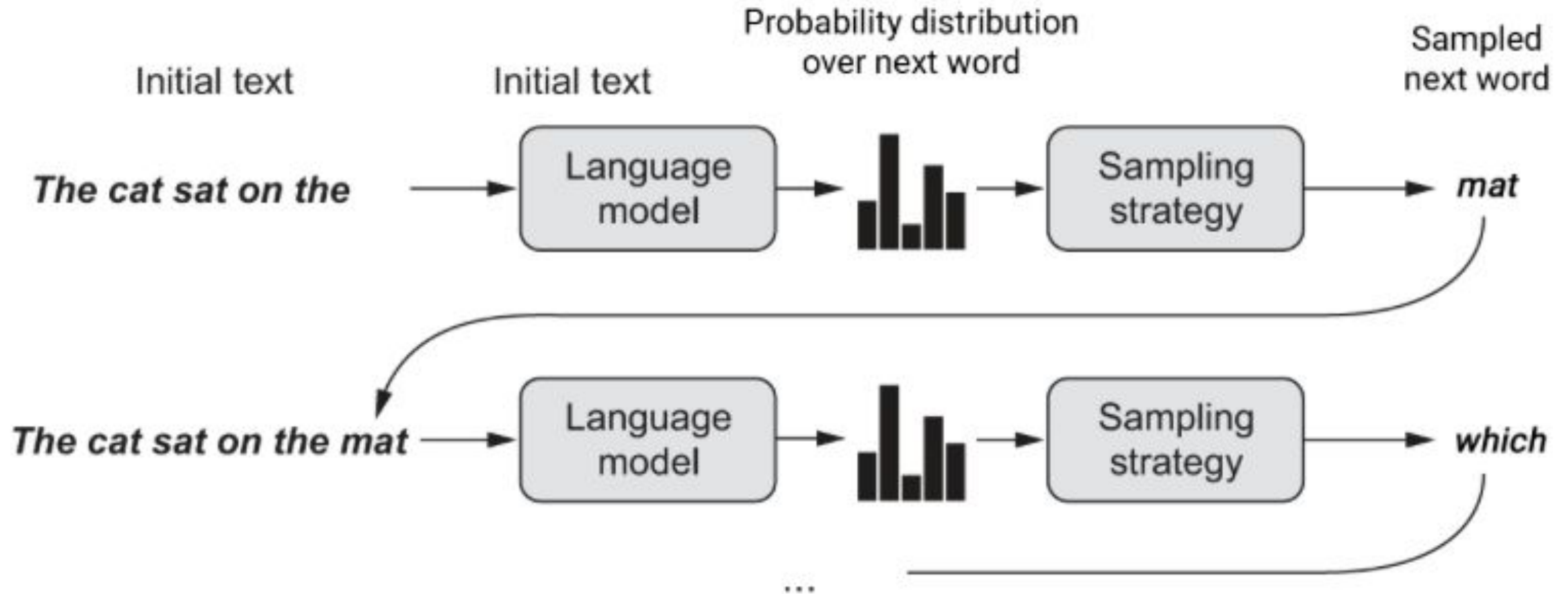
Open

Closed

2026

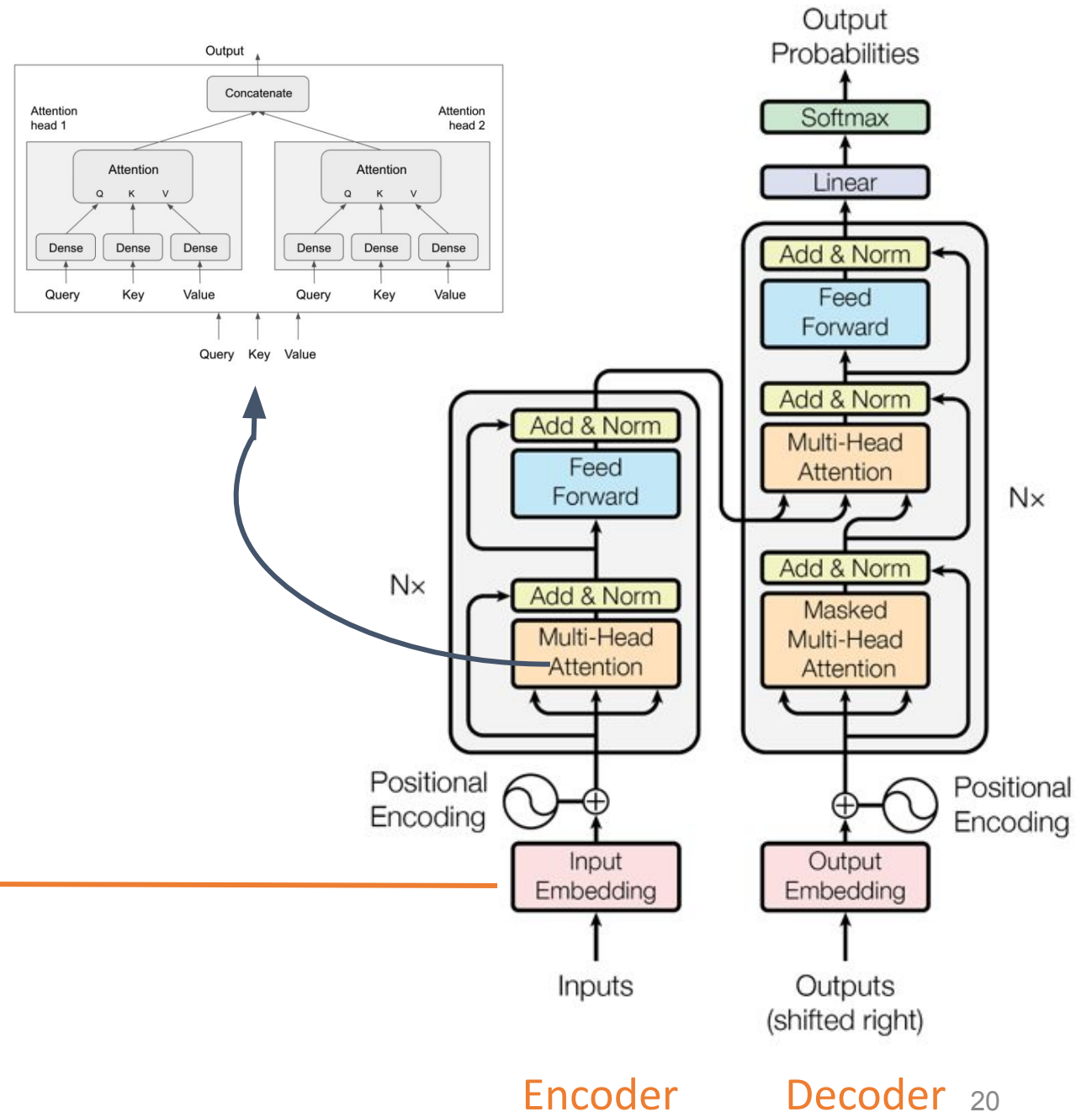
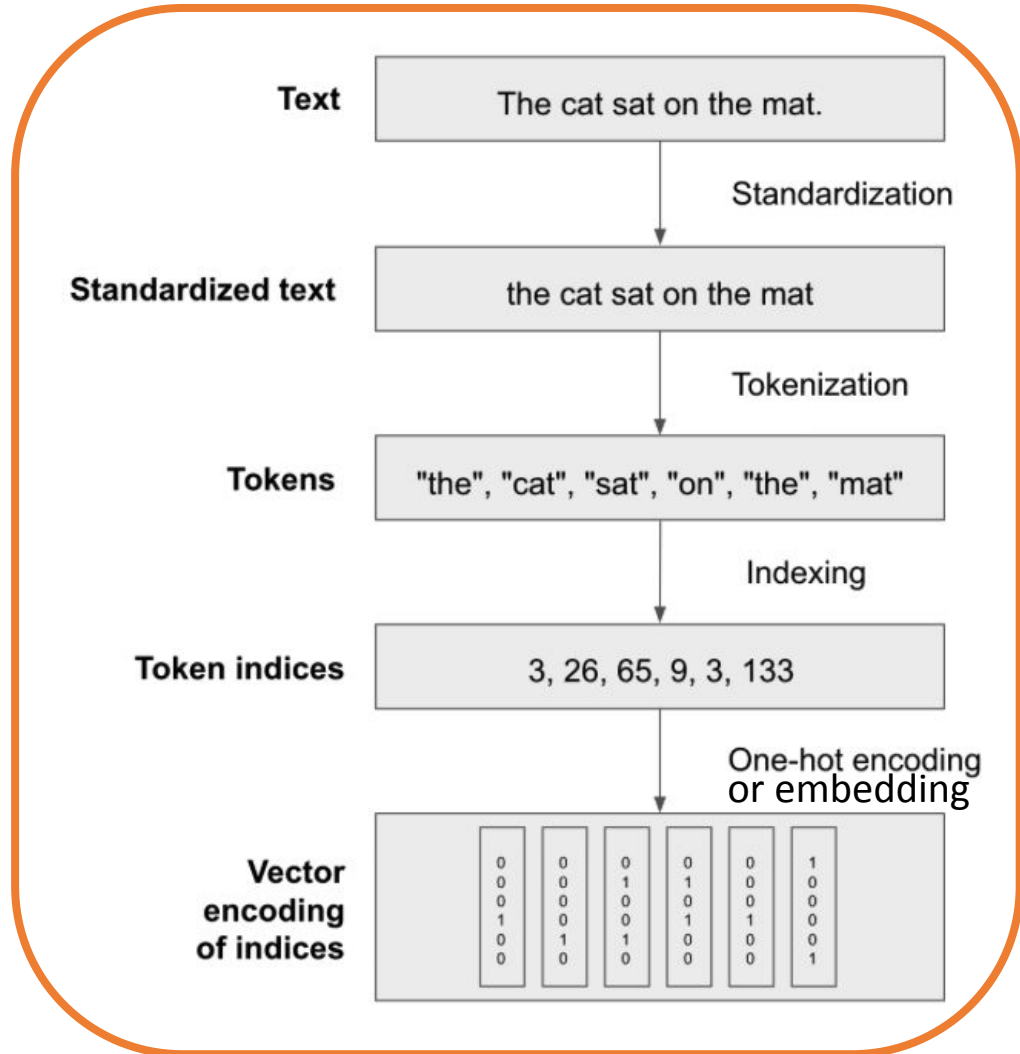


LLM/SLM – Large /Small Language Model

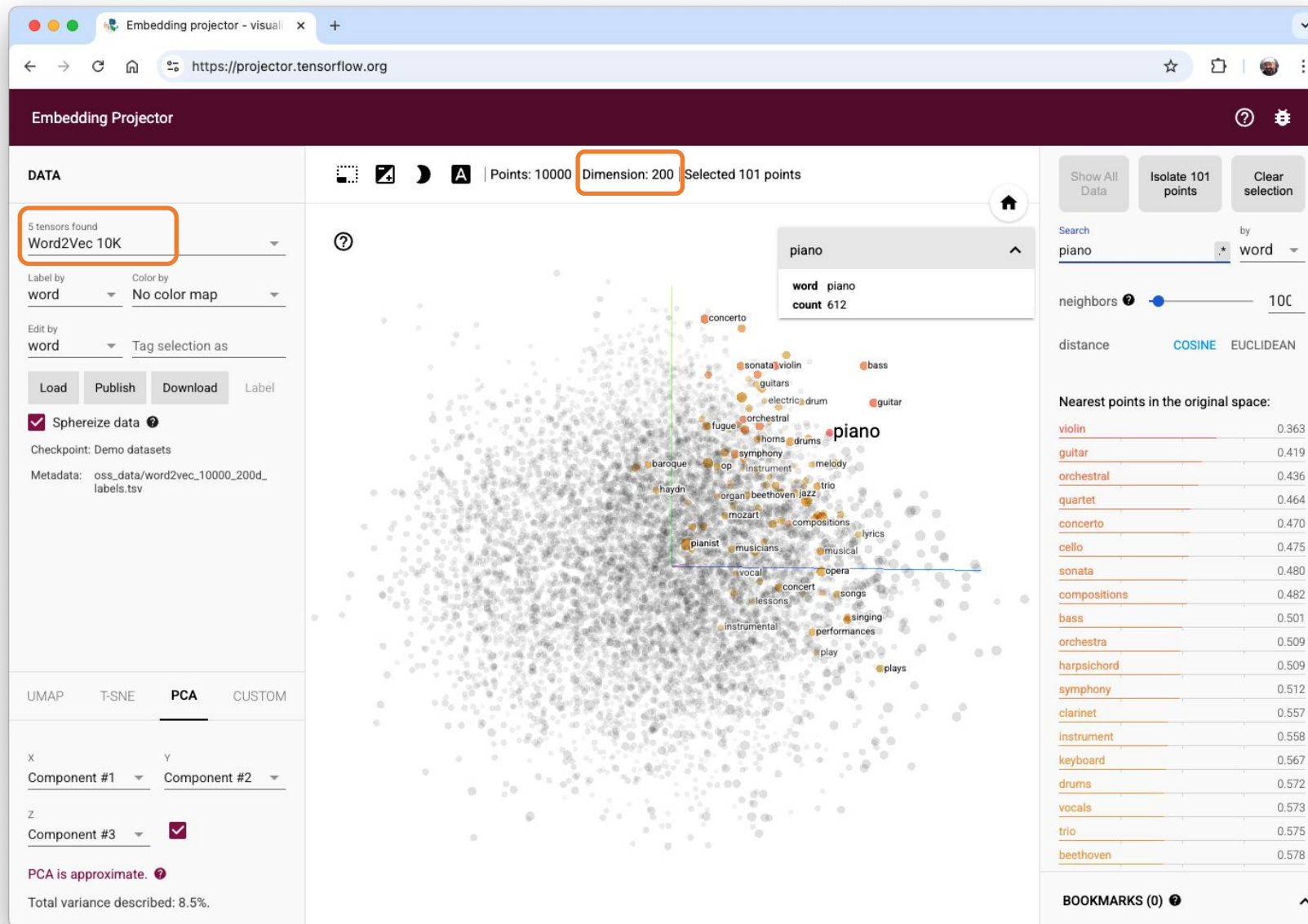


<https://platform.openai.com/tokenizer>

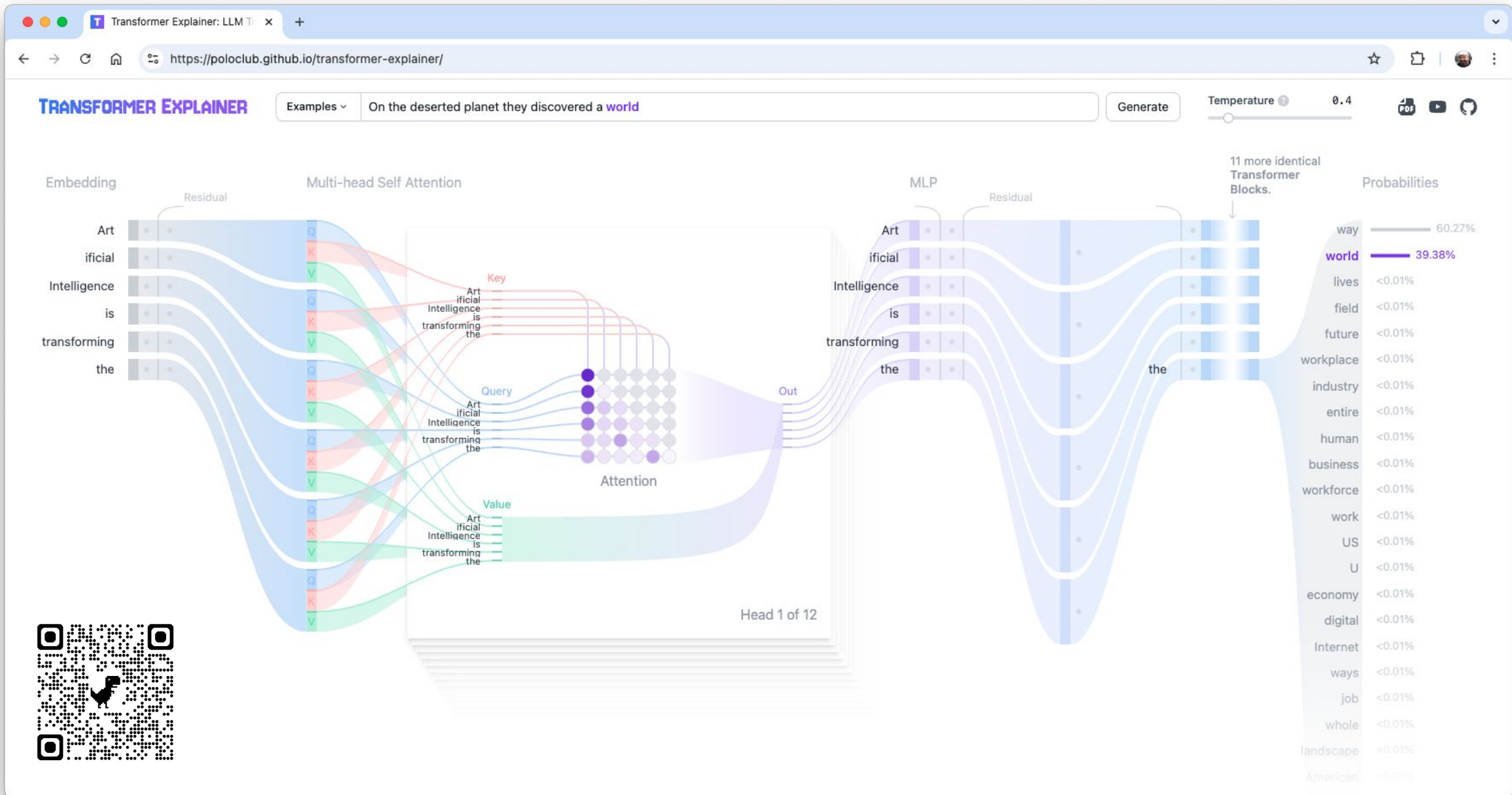
Embedding



Embedding



<https://projector.tensorflow.org/>



LLM TRAINING

Pretraining

Base model

GPT

Post-Training
Supervised
Fine Tuning

SFT model

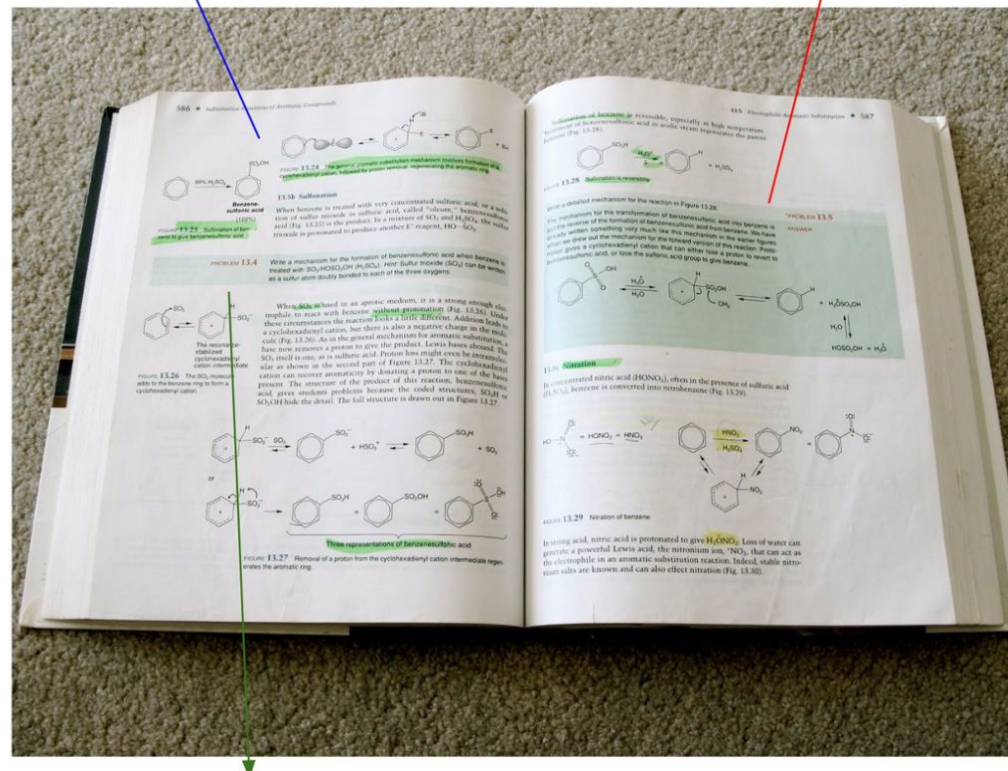
Post-Training
Reinforcement
Learning

RL model

ChatGPT

exposition $\Leftrightarrow$ pretraining
(background knowledge)

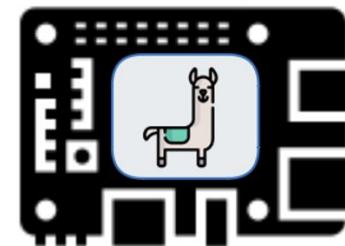
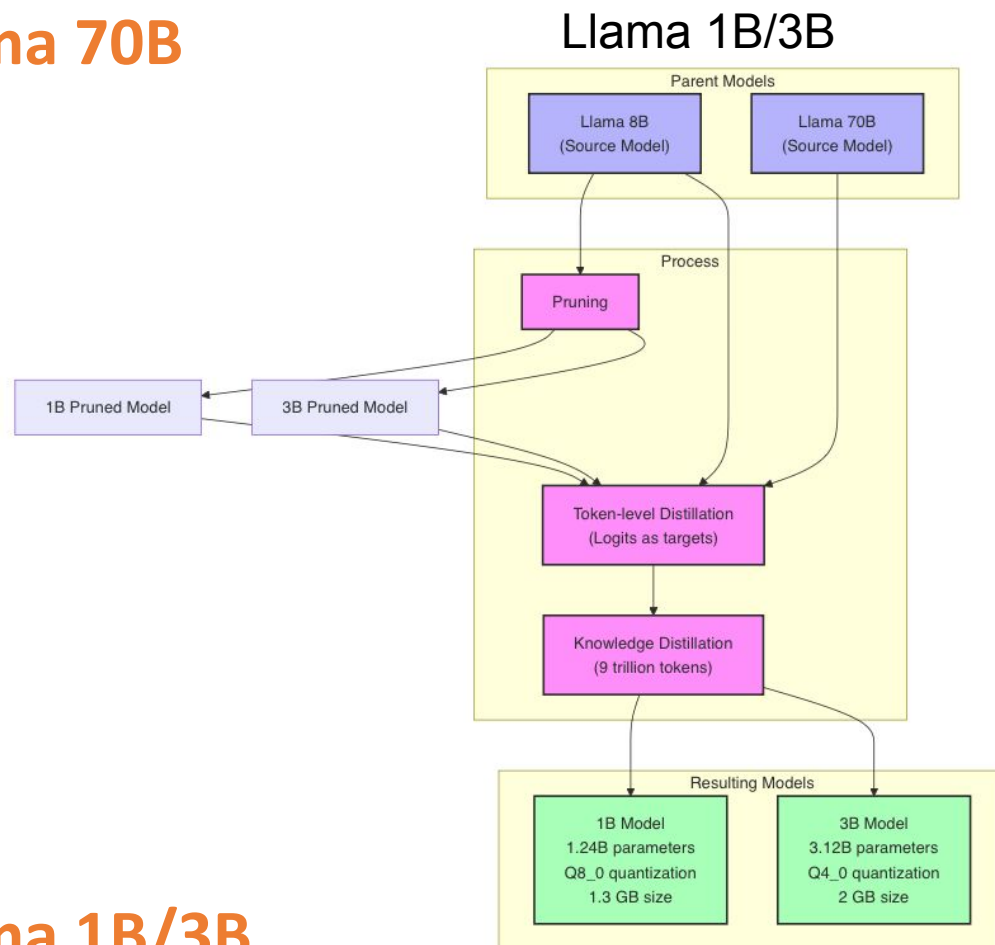
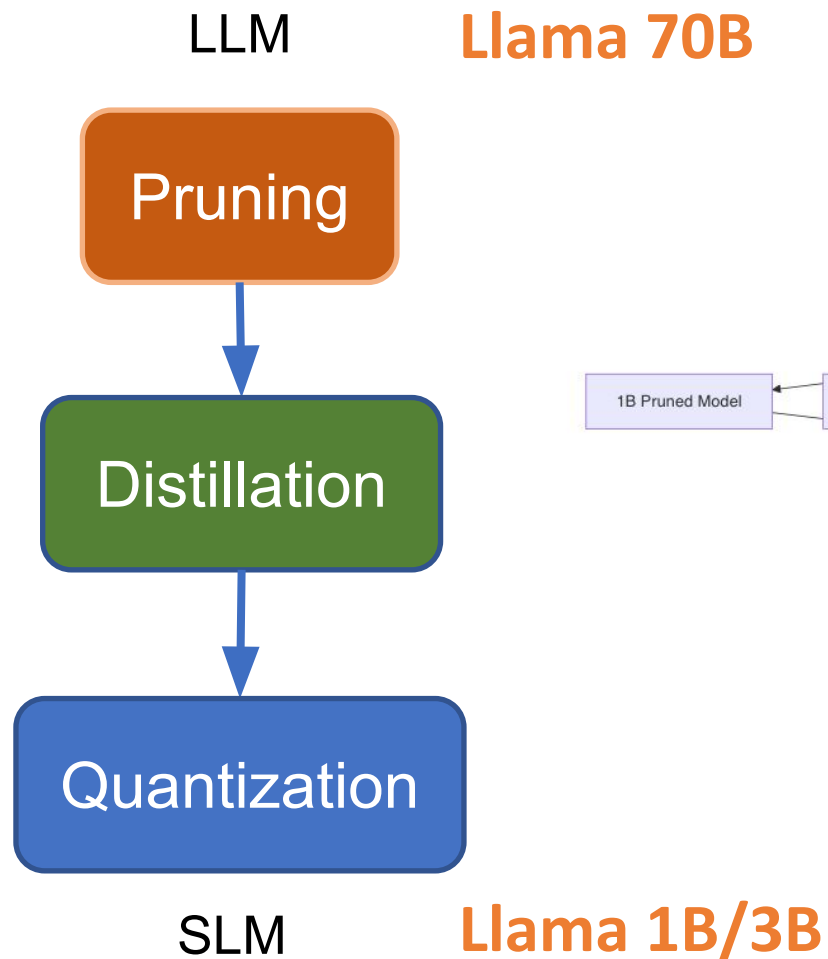
worked problems $\Leftrightarrow$ supervised finetuning
(problem + demonstrated solution, for imitation)



practice problems $\Leftrightarrow$ reinforcement learning
(prompts to practice, trial & error until you reach the correct answer)

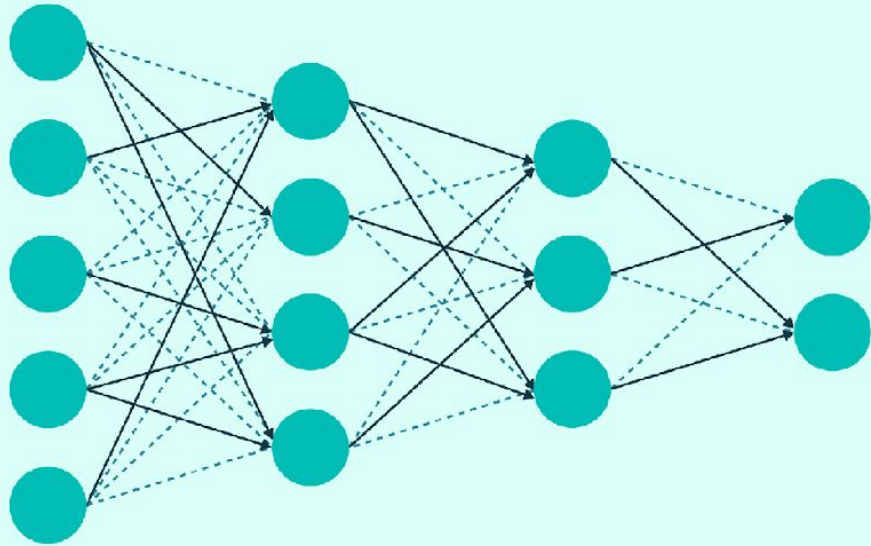
Deep Dive into LLMs like ChatGPT

Small Language Models (SLMs)

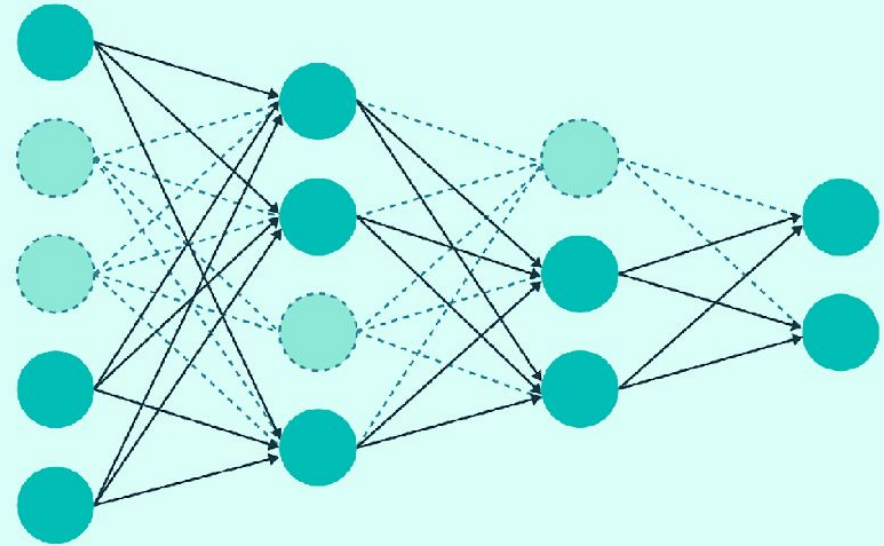


Pruning

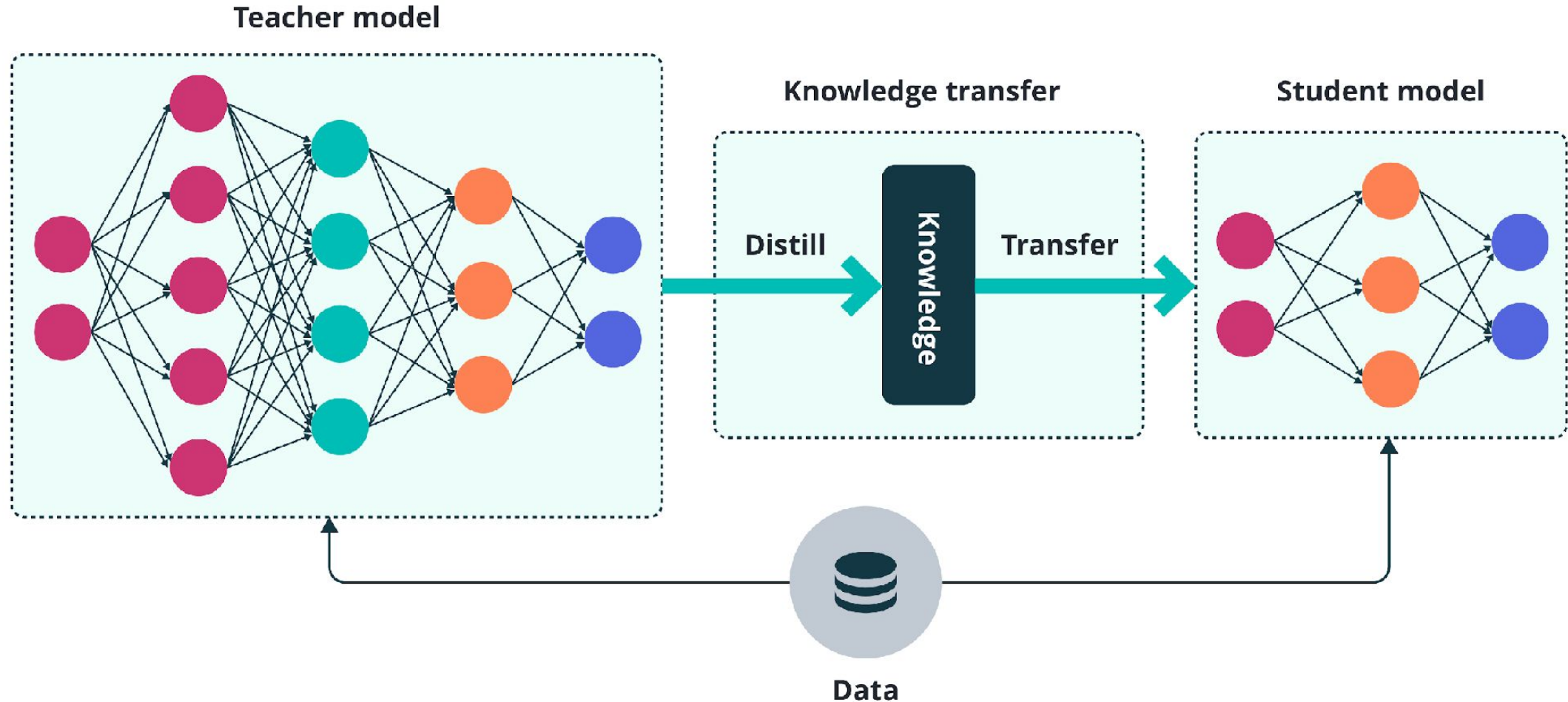
Unstructured pruning



Structured pruning



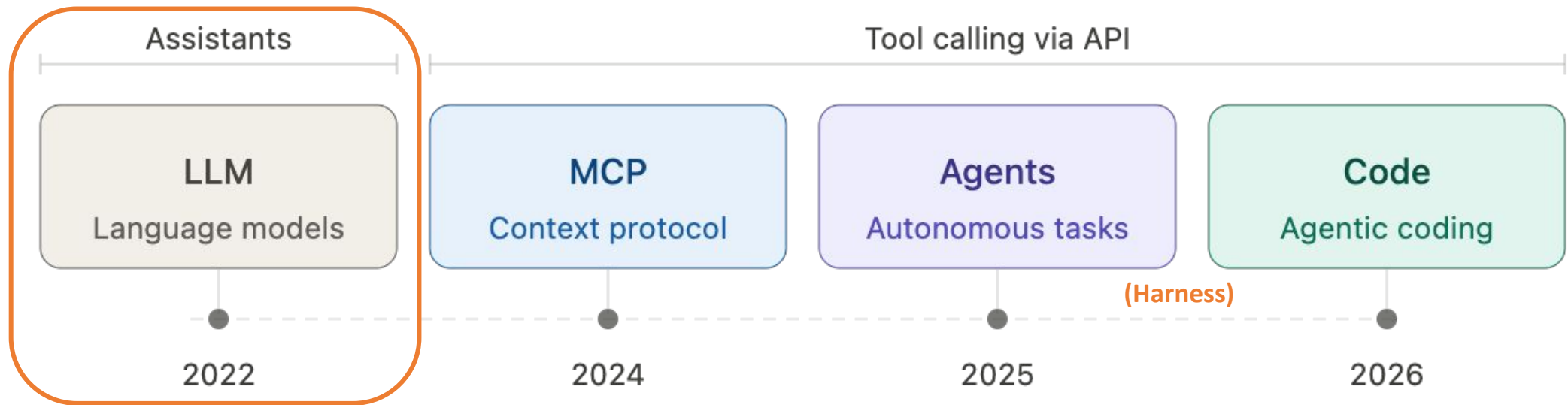
Knowledge Distillation (KD)



Running SLMs locally

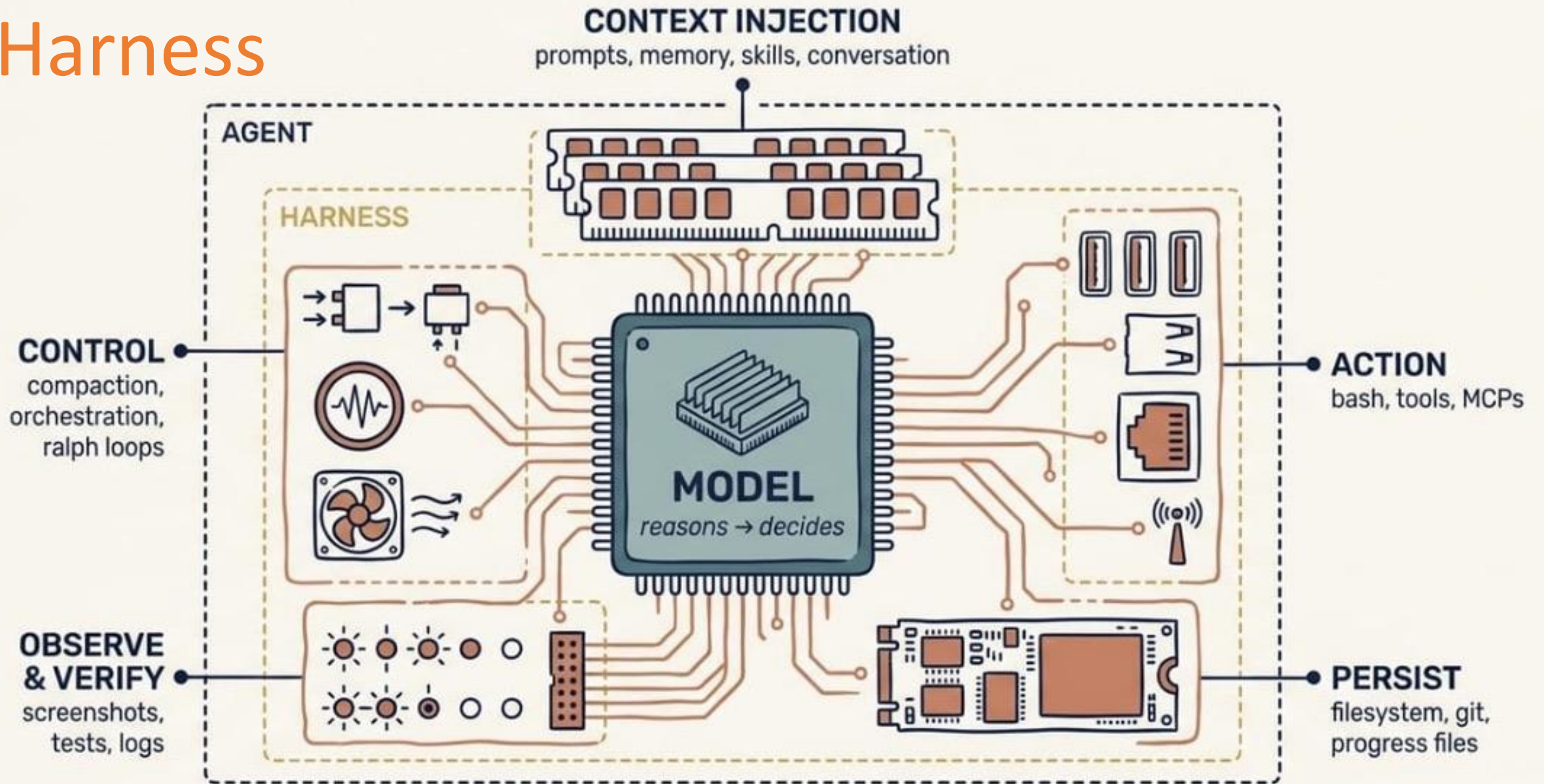
The evolution of AI systems

In just four years, the way we interact with and deploy AI has undergone a structural shift — not just in capability but in the very nature of these systems.



The two eras on the diagram (*Assistants* and *Tool calling via API*) tell the underlying story: early systems were primarily *assistants* — they **augmented human knowledge**. Today's systems operate via *tool-calling APIs* — they extend into the world, use its infrastructure, and take actions with real consequences.

Harness



The model is one chip on the board. The harness is everything else that makes it useful.

How to Run SLMs at the Edge

Ollama

Open-source framework for running SLMs locally. The easiest way to get started on Raspberry Pi — supports Qwen, Phi, Gemma, Llama, LLaVa, and more. Does not run easily on Arduino UNO-Q.

llama.cpp

Lightweight C++ inference engine for quantized models. Tested successfully on Arduino UNO-Q — preferred for more control, compact size, and low memory usage. Also efficient on Raspberry Pi.

HF Transformers

Python library with broad model and task coverage via the Hugging Face ecosystem. Well-supported for running SLMs on Raspberry Pi.

LiteRT-LM

Google AI Edge solution designed for production deployments. Optimized for running language models efficiently across edge devices.

llama.cpp on the Arduino UNO-Q

Running local LLMs and VLMs



CONSTRAINTS

What's realistic on a 4 GB UNO Q (16GB eSSD)

~3.0 GB

RAM free after boot

~1.1 GB

model + 1024 ctx (0.8B Q8) text only
(For vision, ctx should increase to 4096)

4× A53

Cortex cores @ 2 GHz, CPU-only

9.8 GB

single root partition (older images)



Mind the board variant. The 2 GB UNO Q supports only the smallest models (such as the SmolLM2-135M/360M). The 0.8B-parameter models need the 4 GB board.

No NPU for inference. The QRB2210's GPU is not a reliable llama.cpp target — four CPU cores do all the work. That single fact shapes every decision that follows.



STEP 1

Prepare the Linux side

```
marcelo_rovai — arduino@uno-q-4b: ~ — ssh arduino@192.168.5.114 — 82x5  
arduino@uno-q-4b:~$ free -h  
              total        used        free      shared  buff/cache   available  
Mem:           3.6Gi        641Mi        1.1Gi         38Mi         2.0Gi         3.0Gi  
Swap:          1.8Gi         3.1Mi        1.8Gi  
arduino@uno-q-4b:~$
```

✓ **Swap is a safety net.** It absorbs the brief spike while the model loads. You don't want to hit it during inference.

```
# verify the factory swap is present (~1.8 GB)  
free -h  
  
# install the build toolchain  
sudo apt update  
sudo apt install -y build-essential cmake git \  
                    pkg-config libcurl4-openssl-dev
```

✓ All installations must be done in the `/home/arduino`



STEP 2

Build llama.cpp — CMake

```
# shallow clone keeps it ~80-100 MB
cd /home/arduino
git clone --depth 1 https://github.com/ggml-
org/llama.cpp

cd llama.cpp

cmake -B build \
  -DLLAMA_CURL=ON \
  -DGGML_NATIVE=ON \
  -DCMAKE_BUILD_TYPE=Release

cmake --build build --config Release -j4
```

- **LLAMA_CURL=ON** — download models from inside the binary.
- **GGML_NATIVE=ON** — emit the A53's NEON SIMD intrinsics.
- **-j4** — all four cores; ~20–25 min. Drop to -j2 if it runs out of memory.



STEP 3

Pick a model: Qwen3.5-0.8B

Model	Size	Notes
SmolLM2-360M Q4_K_M	~230 MB	(For UNO-Q 2GB only)
Qwen3.5-0.8B Q8_0	~880 MB	Primary choice — best quality-per-parameter
Qwen3.5-2B UD-Q4_K_XL	~1.25 GB	Better quality, ~2× the latency



Q4 vs Q8 for sub-1B models

At 0.8B there's little redundancy, so 4-bit compression bites hard. Prefer Q8_0 or the Unsloth UD-Q4_K_XL dynamic quant, and use min_p over top_p for sampling.

	397B-A17B	122B-A10B	35B-A3B	27B	9B	4B	2B	0.8B
Overall	100%	97%	95%	97%	90%	85%	72%	54%
Text Overall	100%	97%	94%	97%	90%	84%	64%	41%
Instruction Following	100%	97%	93%	98%	88%	82%	63%	37%
Multilingualism	100%	98%	95%	98%	90%	84%	64%	46%
Knowledge	100%	98%	96%	97%	92%	88%	73%	49%
Video Understanding	100%	99%	97%	98%	94%	90%	78%	62%
General VQA	100%	97%	96%	95%	92%	88%	81%	68%
Text Recognition and Document Understanding	100%	98%	98%	98%	95%	92%	85%	71%

Why Qwen3.5?

- ✓ Native vision-language model
- ✓ Tools
- ✓ Reasoning (thinking)
- ✓ 201 languages
- ✓ Apache 2.0.



Download model and RUN · CLI

First tokens with llama-cli (text generation)

```
wget
https://huggingface.co/unsloth/Qwen3.5-0.8B-GGUF/resolve/main/
Qwen3.5-0.8B-Q8_0.gguf

wget
https://huggingface.co/unsloth/Qwen3.5-0.8B-GGUF/resolve/main/mm
proj-F16.gguf \
-O Qwen3.5-0.8B-mmproj-F16.gguf
```

```
./build/bin/llama-cli \
--model ~/models/Qwen_Qwen3.5-0.8B-Q8_0.gguf \
--threads 4 --ctx-size 1024 \
--temperature 0.7 --top-p 0.8 --top-k 20 \
--presence-penalty 1.5 \
--reasoning off --reasoning-budget 0
```

Download model and projector:

- ✓ model.gguf
- ✓ mmproj.gguf

- ✓ **temperature** 0.7
- ✓ **top_p** 0.8
- ✓ **top_k** 20
- ✓ **presence_penalty** 1.5 — stops repetition loops



Disable thinking mode. --reasoning off --reasoning-budget 0. Without it the model burns minutes on internal chains and often loops. The old --chat-template-kwarg flag is deprecated.

5–10 tok/s

CLI throughput, usable

No Thinking

marcelo_rovai — arduino@uno-q-4b: ~/llama.cpp — ssh arduino@192.168.5.114 — 67x37

```
arduino@uno-q-4b:~/llama.cpp$ ./build/bin/llama-cli \  
--model ~/models/Qwen_Qwen3.5-0.8B-Q8_0.gguf \  
--threads 4 --ctx-size 1024 \  
--temperature 0.7 --top-p 0.8 --top-k 20 \  
--presence-penalty 1.5 \  
--reasoning off --reasoning-budget 0
```

Loading model...



```
build      : b9330-328874d05  
model      : Qwen_Qwen3.5-0.8B-Q8_0.gguf  
modalities : text
```

```
available commands:  
/exit or Ctrl+C    stop or exit  
/regen             regenerate the last response  
/clear             clear the chat history  
/read <file>       add a text file  
/glob <pattern>    add text files using globbing pattern
```

> What is the capital of Ecuador? Answer with only one word.

Quito

[Prompt: 10.3 t/s | Generation: 7.8 t/s]

> █

Thinking

marcelo_rovai — arduino@uno-q-4b: ~/llama.cpp — ssh arduino@192.168.5.114 — 67x37



```
build      : b9330-328874d05  
model      : Qwen_Qwen3.5-0.8B-Q8_0.gguf  
modalities : text
```

```
available commands:  
/exit or Ctrl+C    stop or exit  
/regen             regenerate the last response  
/clear             clear the chat history  
/read <file>       add a text file  
/glob <pattern>    add text files using globbing pattern
```

> hello

[Start thinking]

Okay, the user just said "hello". Hmm, they're greeting me. Let's see, I need to respond in a friendly manner. Since this is a simple hello, I should be warm and engaging. Maybe ask if they have something interesting to discuss or if there's anything specific you'd like help with. Keep it concise but inviting. Let's make sure to include an emoji or something that makes it more approachable. Alright, responding in a friendly tone now.

[End thinking]

Hello! How can I assist you today? 😊

[Prompt: 9.0 t/s | Generation: 5.3 t/s]

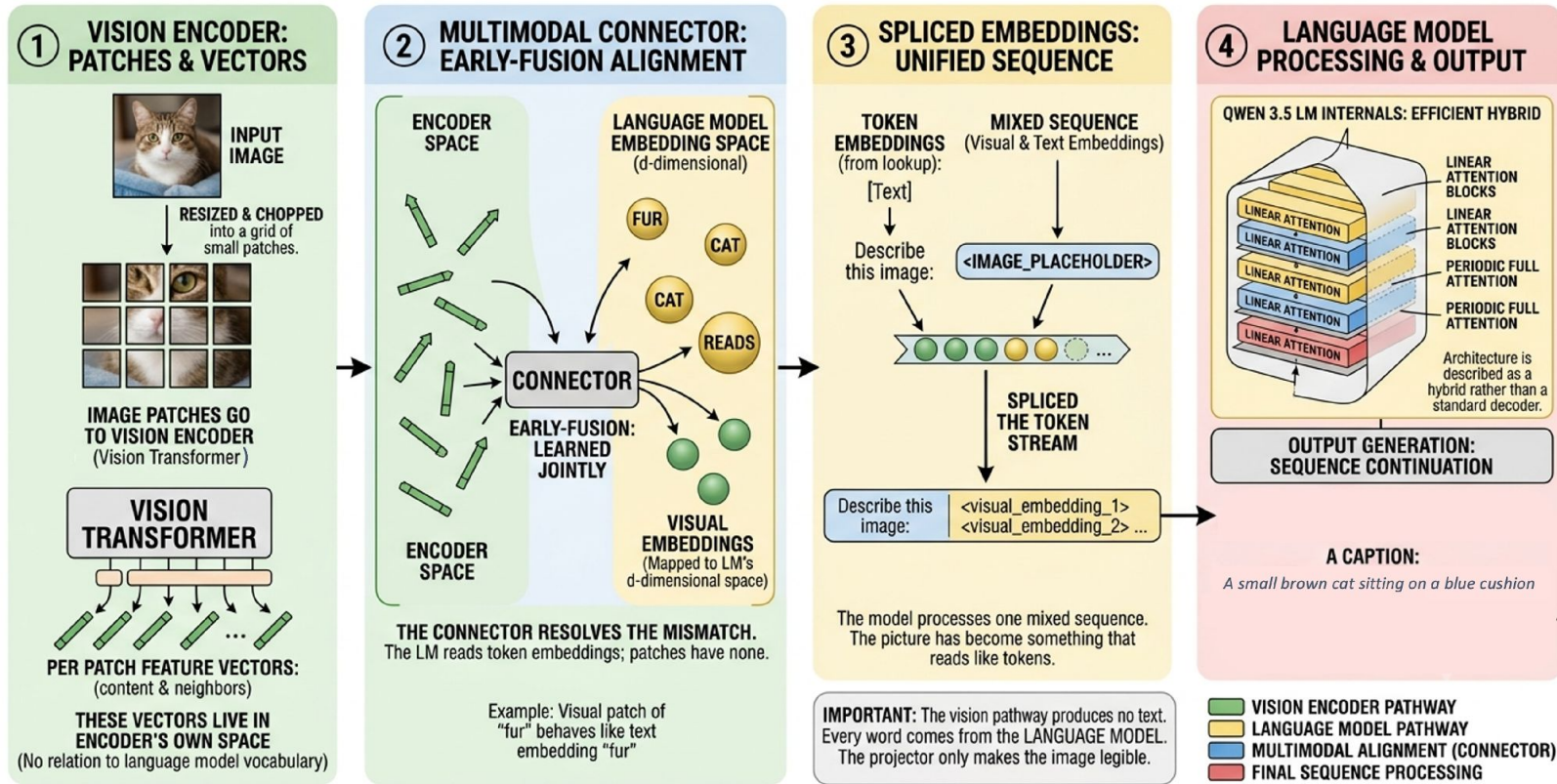
> █



VISION

One model, plus a projector

HOW QWEN 3.5 UNDERSTANDS IMAGES: VISION-LANGUAGE ENCODING AND EARLY FUSION



✓ Add one file — the mmproj. The same Qwen3.5 weights describe an image; the projector holds the vision encoder + connector.

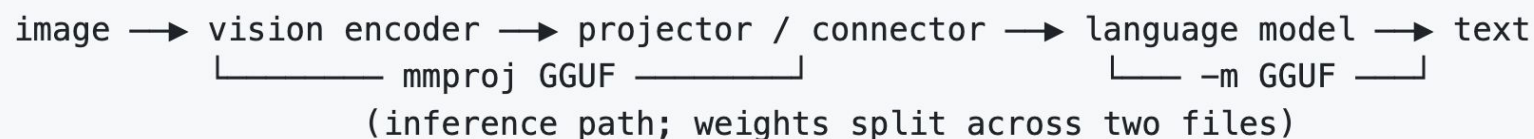
✓ Early fusion, not a bolt-on. Trained on text and image tokens together. Pair the mmproj with the same base — n_embd must match.

Prefer the F16 projector. The A53 has no native BF16 path. Switching BF16 → F16 cut a 640² describe from 4 min to ~1 min.



SERVE · VISION

Flags that matter



Use images smaller than 640×640.

```
./build/bin/llama-server \  
-m      .../Qwen_Qwen3.5-0.8B-Q8_0.gguf \  
--mmproj .../mmproj-F16_0.8B.gguf \  
--host 0.0.0.0 --port 8081 \  
-c 4096 -t 4 -n 1024 \  
--jinja \  
--image-max-tokens 256 \  
--reasoning off --reasoning-budget 0
```

- ✓ **--jinja** — loads Qwen's chat template so the stop token is registered. Skip it and generation never stops.
- ✓ **-c 4096** — one image is hundreds–thousands of tokens; 1024 overflows before the prompt.
- ✓ **--image-max-tokens 256** — caps the encoder's work, the single biggest speed lever.

```

arduino@uno-q-4b:~/llama.cpp$ ./build/bin/llama-server \
-m /home/arduino/models/Qwen_Qwen3.5-0.8B-Q8_0.gguf \
--mmproj /home/arduino/models/mmproj-F16_0.8B.gguf \
--host 0.0.0.0 --port 8081 \
-c 4096 -t 4 \
-n 256 \
--jinja \
--image-max-tokens 256 \
--reasoning off \
--reasoning-budget 0

```

HW Configuration

```

0.00.011.656 I log_info: verbosity = 3 (adjust with the `-lv N` CLI arg)
0.00.011.677 I device_info:
0.00.011.743 I   - CPU      : CPU (3667 MiB, 3667 MiB free)
0.00.012.075 I system_info: n_threads = 4 (n_threads_batch = 4) / 4 | CPU : NEON = 1 | ARM_FMA = 1 | LLAMAFILE = 1 | OPENMP = 1 | REPACK = 1 |
0.00.012.099 I srv llama_server: n_parallel is set to auto, using n_parallel = 4 and kv_unified = true
0.00.012.399 I srv      init: running without SSL
0.00.012.580 I srv      init: using 8 threads for HTTP server
0.00.013.100 I srv      start: binding port with default address family

```

Loading and starting the model

```

0.00.014.701 I srv llama_server: loading model
0.00.014.729 I srv load_model: loading model '/home/arduino/models/Qwen_Qwen3.5-0.8B-Q8_0.gguf'
0.00.194.048 I srv load_model: [mtdm] estimated worst-case memory usage of mmproj is 356.79 MiB
0.00.194.129 I common_init_result: fitting params to device memory ...
0.00.194.131 I common_init_result: (for bugs during this step try to reproduce them with -fit off, or provide --verbose logs if the bug only occurs with -fit on)
0.03.377.389 I common_params_fit_impl: projected to use 1402 MiB of host memory vs. 3667 MiB of total host memory
0.06.015.219 W llama_context: n_ctx_seq (4096) < n_ctx_train (262144) -- the full capacity of the model will not be utilized
0.06.084.292 I common_init_from_params: warming up the model with an empty run - please wait ... (--no-warmup to disable)
0.06.379.759 W load_hparams: Qwen-VL models require at minimum 1024 image tokens to function correctly on grounding tasks
0.06.379.775 W load_hparams: if you encounter problems with accuracy, try adding --image-min-tokens 1024
0.06.379.776 W load_hparams: more info: https://github.com/ggml-org/llama.cpp/issues/16842

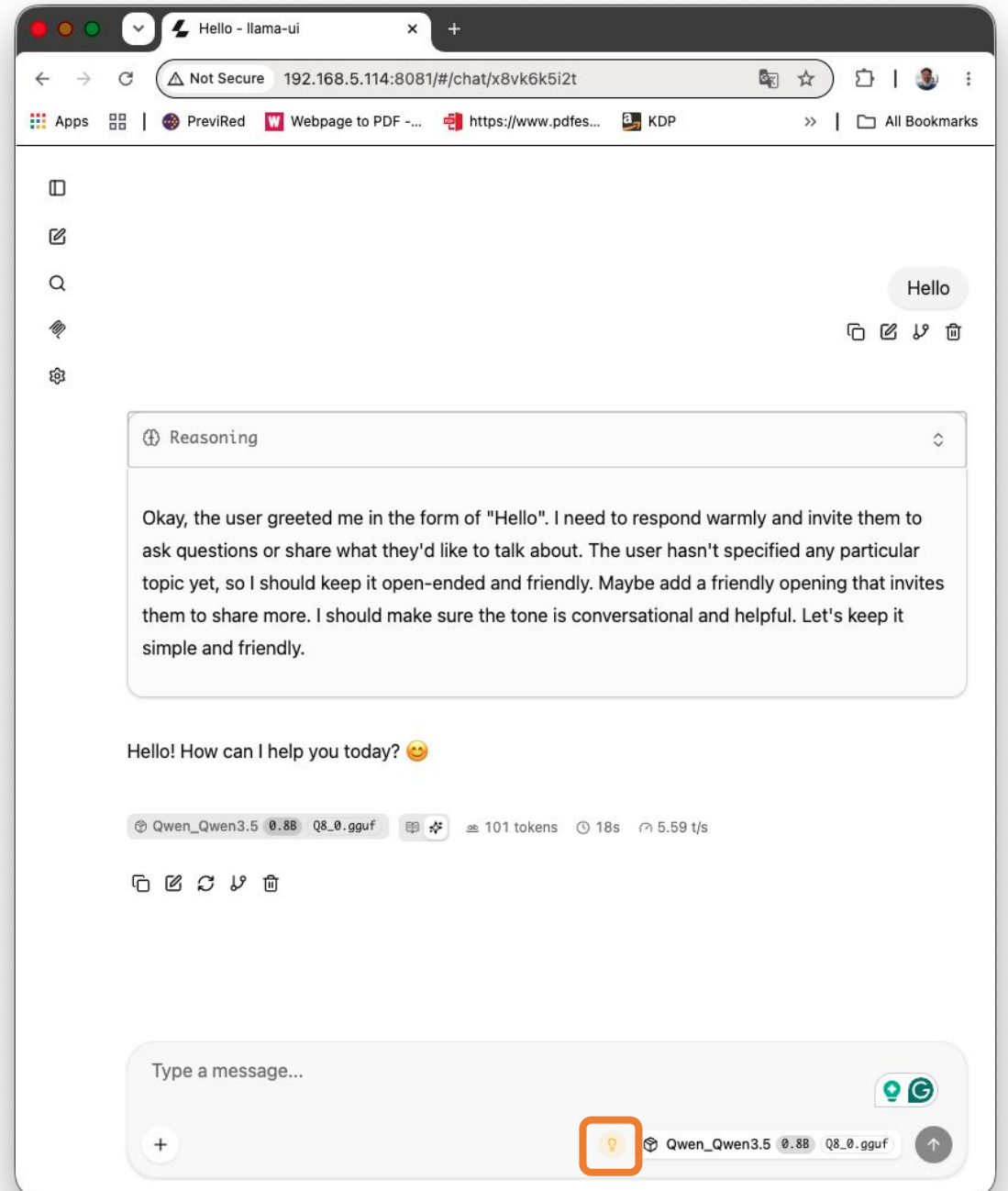
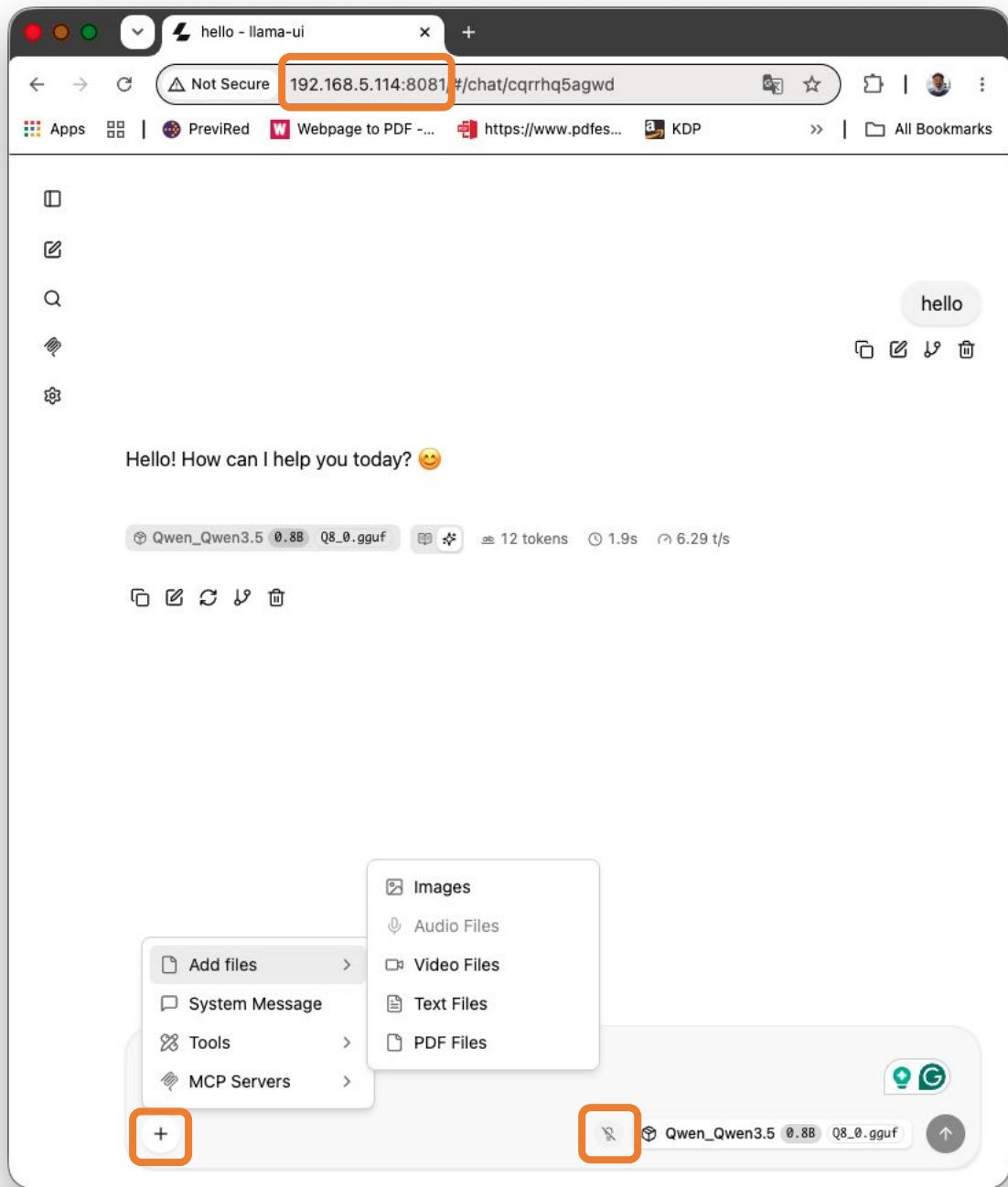
```

Loading and starting the projector

```

0.06.518.008 I srv load_model: loaded multimodal model, '/home/arduino/models/mmproj-F16_0.8B.gguf'
0.06.518.057 I srv load_model: initializing slots, n_slots = 4
0.06.807.845 W srv load_model: speculative decoding will use checkpoints
0.06.807.895 W common_speculative_init: no implementations specified for speculative decoding
0.06.807.899 I slot load_model: id 0 | task -1 | new slot, n_ctx = 4096
0.06.807.914 I slot load_model: id 1 | task -1 | new slot, n_ctx = 4096
0.06.807.916 I slot load_model: id 2 | task -1 | new slot, n_ctx = 4096
0.06.807.917 I slot load_model: id 3 | task -1 | new slot, n_ctx = 4096
0.06.808.238 I srv load_model: prompt cache is enabled, size limit: 8192 MiB
0.06.808.247 I srv load_model: use '--cache-ram 0' to disable the prompt cache
0.06.808.248 I srv load_model: for more info see https://github.com/ggml-org/llama.cpp/pull/16391
0.06.808.250 I srv load_model: context checkpoints enabled, max = 32, min spacing = 256

```



Describe the image in one paragraph



Describe the image in one paragraph

This serene rural image captures a breathtaking, overgrown mango tree with a wooden treehouse perched high above its trunk, surrounded by lush greenery and scattered fruit. Two children are nestled in the treehouse, one standing and the other sitting, both looking out toward the tranquil countryside beyond. A rustic wooden ladder winds down the side of the massive, sprawling tree, adding to the sense of ancient nature. Below, a dirt road stretches into the distance under a bright blue sky, flanked by rolling hills and distant mountains, all bathed in warm sunlight that highlights the vibrant greens and earth tones of the landscape.

Qwen_Qwen3.50.88Q8_0.gguf124 tokens22s5.43 t/s

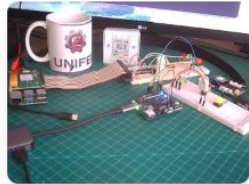
Type a message...

+

Qwen_Qwen3.50.88Q8_0.gguf

↑

Describe the image in one sentence



Describe the image in one sentence

An electronics project with a Raspberry Pi and breadboard components is set up on a green cutting mat with a mug and a timer nearby.

Qwen_Qwen3.50.88Q8_0.gguf28 tokens5.0s5.57 t/s

Type a message...

+

Qwen_Qwen3.50.88Q8_0.gguf

↑

What is written on the mug?

UNIFE

Qwen_Qwen3.50.88Q8_0.gguf3 tokens0.4s8.05 t/s

Type a message...

+

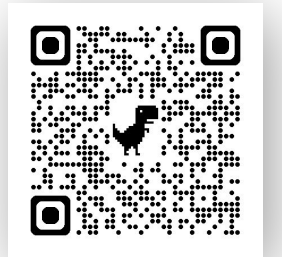
Qwen_Qwen3.50.88Q8_0.gguf

↑

42



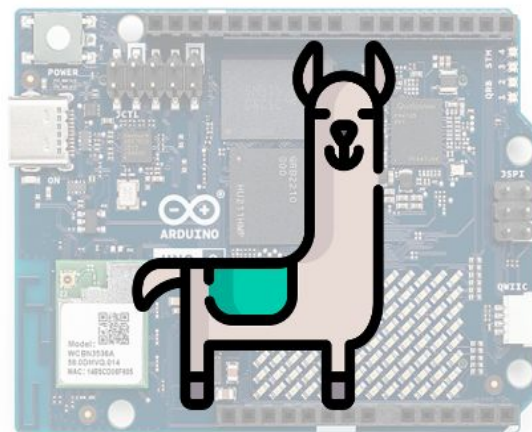
VisText-Mosquito Project





SERVE · VISION

VisText-Mosquito Project



Qwen3.5-0.8B

Question: *Does this image depict a potential mosquito breeding site?*

Response: *Yes*

Reasoning: *The image shows a tire lying on the ground with a large hole in the rubber, providing a suitable breeding spot for mosquitoes.*

You are creating a dataset for

192.168.5.114:8081/#/chat/xgj042pgesf

Apps | PreviRed | Webpage to PDF -... | https://www.pdfes... | KDP | All Bookmarks



You are creating a dataset for an image analysis model designed to identify potential mosquito breeding sites. The dataset will consist of image-question-answer pairs. For the provided image, generate a question asking whether it depicts a potential mosquito breeding site. Then, provide a detailed answer explaining why or why not. Don't give extra text.

Output format:

Question: [Your generated question]

Response: [Yes/No]

Reasoning(Why): [Your detailed reasoning]

Question: Is this plant likely to be a mosquito breeding site?

Response: Yes

Reasoning: The plant's dense, leafy foliage can trap and attract mosquitoes, and the pot's material and placement are consistent with garden or utility areas where such sites are common.

Qwen_Qwen3.5 0.88 Q8_0.gguf 56 tokens 10s 5.51 t/s

Type a message...

Qwen_Qwen3.5 0.88 Q8_0.gguf

You are creating a dataset for

192.168.5.114:8081/#/chat/yltw94m0gq

Apps | PreviRed | Webpage to PDF -... | https://www.pdfes... | KDP | All Bookmarks



You are creating a dataset for an image analysis model designed to identify potential mosquito breeding sites. The dataset will consist of image-question-answer pairs. For the provided image, generate a question asking whether it depicts a potential mosquito breeding site. Then, provide a detailed answer explaining why or why not. Don't give extra text.

Output format:

Question: [Your generated question]

Response: [Yes/No]

Reasoning(Why): [Your detailed reasoning]

Question: Is the image depicting a potential mosquito breeding site?

Response: Yes

Reasoning(Why):

1. There is a large mug with the word 'UNIFE' and a shield-like logo on it.
2. Next to the mug, there is a digital display showing a temperature of 82 degrees, which is a typical temperature for mosquito breeding conditions (high humidity combined with moderate room temperature).
3. The environment includes a wooden mat with a woven cloth underneath, which is a common mat found in areas where mosquitoes breed in humid conditions, specifically on bamboo mats or similar mats found in tropical areas.
4. The overall setup, including the electronic components and the specific mat, strongly suggests that this is a mosquito breeding site.

Yes

Qwen_Qwen3.5 0.88 Q8_0.gguf 160 tokens 29s 5.43 t/s

Type a message...

Qwen_Qwen3.5 0.88 Q8_0.gguf

45

The user wants me to classify the provided image as a mosquito breeding site or not.

1. Analyze the image:

1. Foreground:

1. Left: A white mug with "UNIFE" on it.
2. Center-left: A breadboard board (likely a breadboard) with jumper cables connected.
3. Center: A blue microcontroller board (Arduino-like).
4. Center-right: A breadboard with wires connected to it (orange, blue, yellow wires).
5. Right: A white PVC pipe structure, possibly a drainage system or water trap, connected to the breadboard.
6. Background: A computer monitor (LG screen).

2. Context: This looks like a DIY electronics project. The components are breadboards, wires, a microcontroller, and what looks like a drainage pipe setup.

2. Evaluate for mosquito breeding sites:

1. Mosquito breeding sites typically involve standing water, stagnant pools, or flooded drains.
2. The image shows a DIY electronic project on a mat. It's not a body of standing water, a flooded drain, or a pool.
3. The components are clearly for electronics (Arduino, breadboards).
4. Therefore, this is not a mosquito breeding site.

3. Formulate the output:

1. **Question:** Is this an image of a potential mosquito breeding site?
2. **Reasoning:** The image depicts a DIY electronics project involving a breadboard, microcontroller, and piping. It does not show any standing water, stagnant pools, or natural breeding environments that would attract mosquitoes.
3. **Format:**
 1. Question: [Text]
 2. Response: [Yes/No]
 3. Reasoning: [Text]

4. Refine the reasoning:

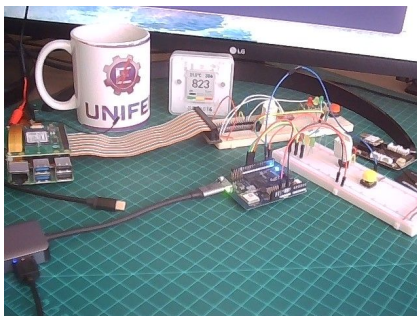
1. The image shows a computer lab setting or a workbench.
2. There are electronics components: breadboards, jumper cables, a microcontroller board (likely an Arduino).
3. There is piping/wiring.
4. There is a mug.
5. There is a monitor.
6. No mosquito breeding site features (standing water, drains, pools) are visible.

5. Final check of constraints: "Don't give extra text."

1. I need

With Reasoning (cap 512 tks)

Output Latency: 1min 52s



Question: Is this an image of a potential mosquito breeding site?

Response: No

Reasoning: This image shows a DIY electronics project on a workbench. It features a breadboard, jumper cables, a microcontroller board (likely an Arduino), and piping for water drainage. There are no visible signs of standing water, pools, or flooded drains typical of mosquito breeding sites.



TRADE-OFF

Fast screen vs quality pass



Qwen3.5-0.8B Q8_0

- **Projector** ~220 MB
- **Decode** 5.42 tok/s · ~1 min / image
- **Quality** good gist, shaky specifics
- **Role** the fast screen



Qwen3.5-2B UD-Q4_K_XL

- **Projector** 668 MB
- **Decode** 2.44 tok/s · ~4 min / image
- **RAM** ~3 GB — fits, swap ~0
- **Role** the quality pass

Two tiers, not a free upgrade — and --image-max-tokens is the dial between them.



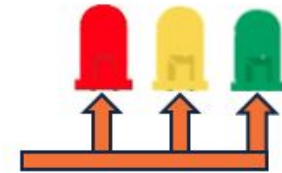
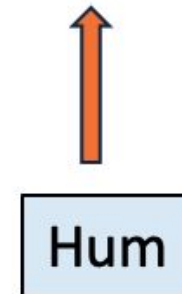
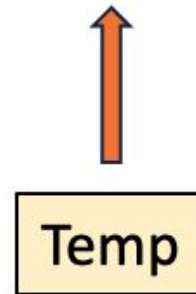
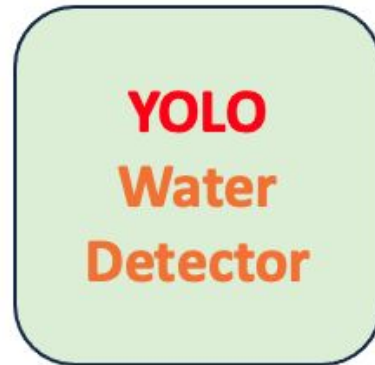
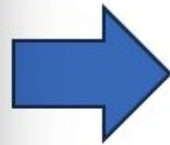
Mosquito Classifier Project

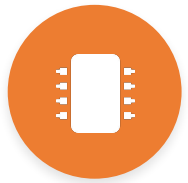




SERVE · INFO TEXT

Mosquito Classifier Project





HARDWARE

Two brains on one board



MPU — Qualcomm QRB2210

4× Cortex-A53 · Debian Linux · runs the SLM + Python



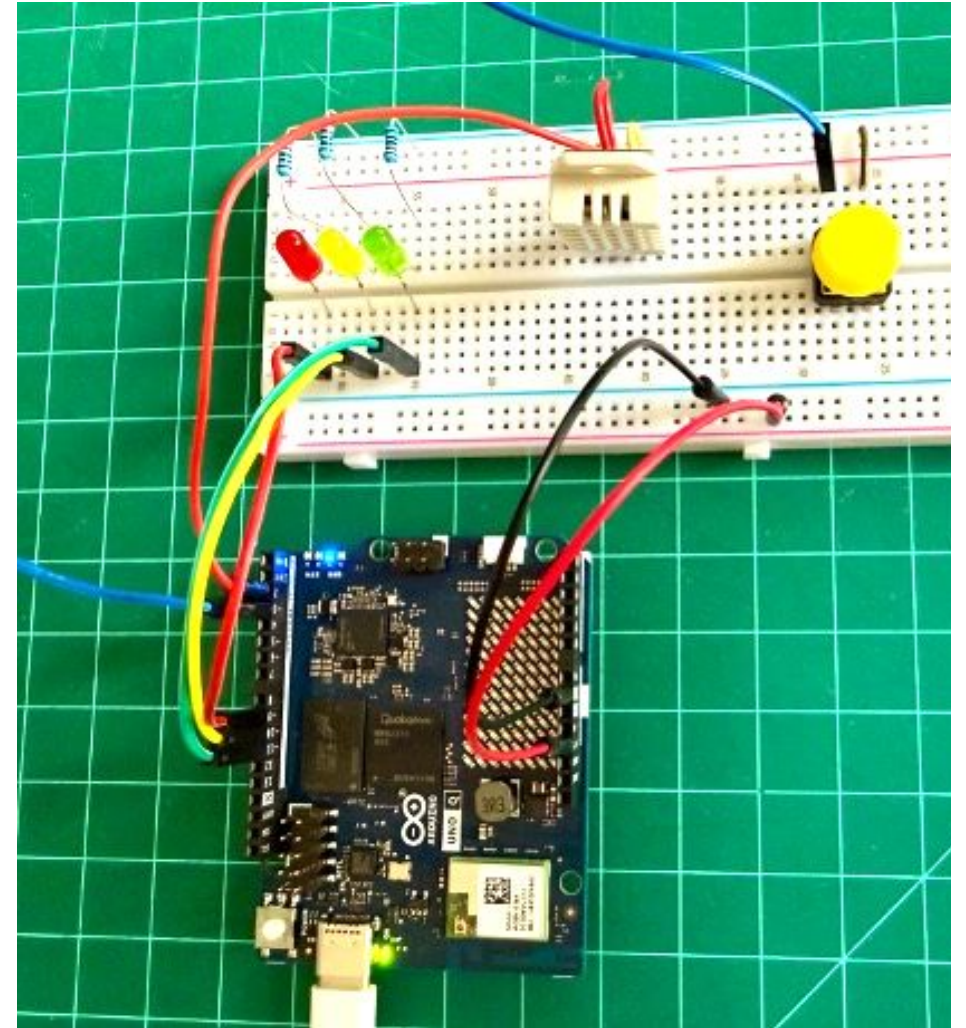
MCU — STM32U585

Zephyr OS · real-time sensing & actuation

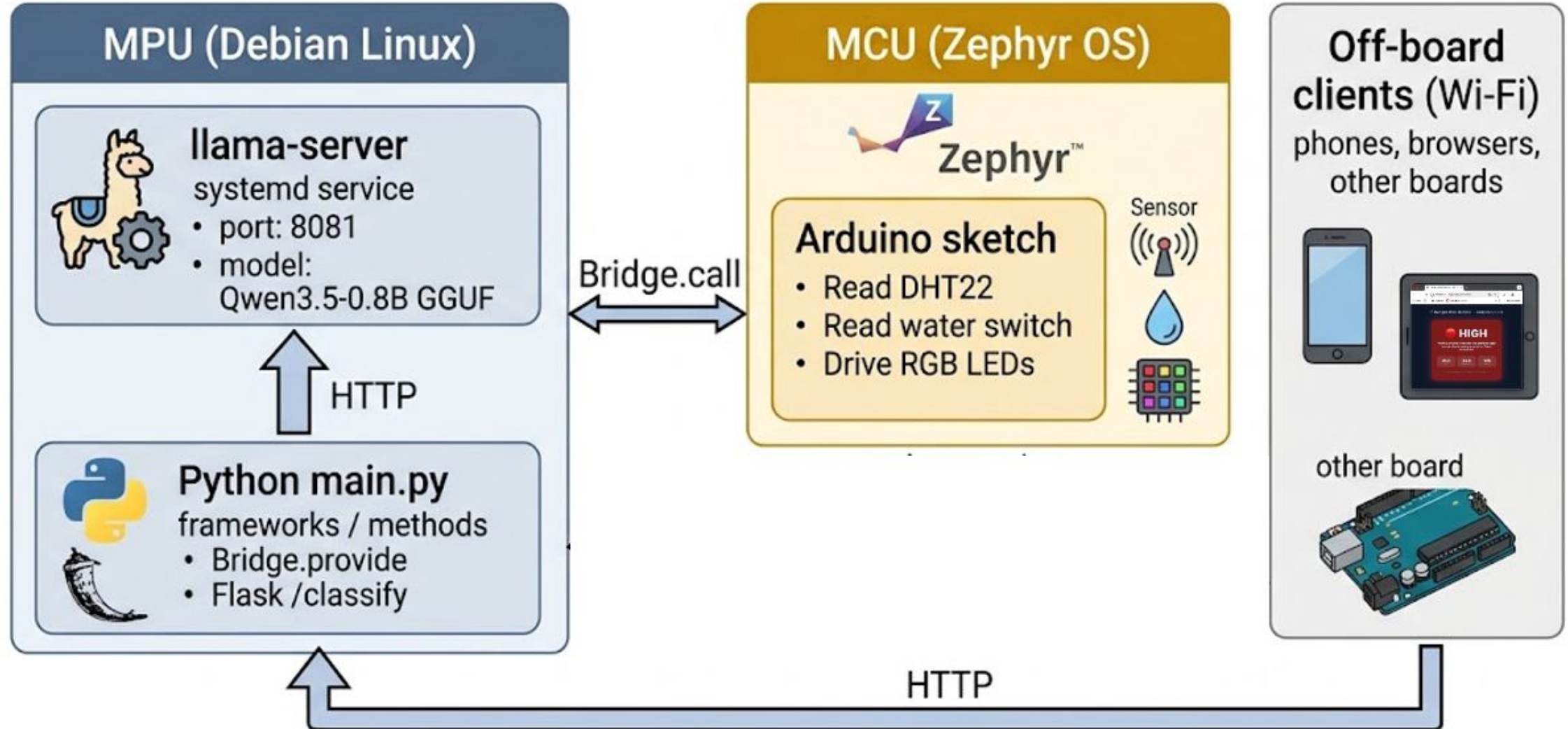


Bridge RPC — the glue between the two processors

You'll also wire: a DHT22 sensor, a water-presence switch, RGB LEDs (220 Ω), a breadboard, and a USB-C cable.

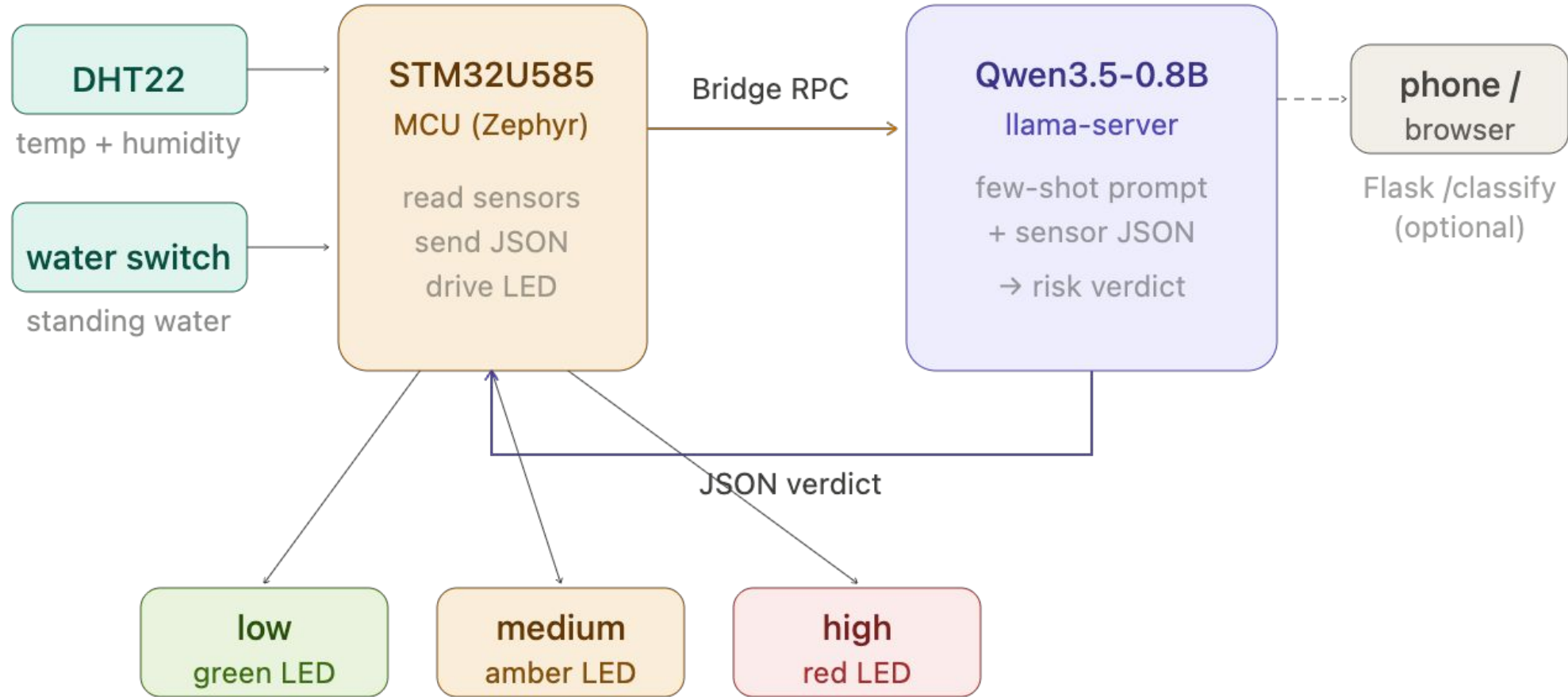


SYSTEM ARCHITECTURE: ARDUINO UNO Q QWEN AI CLASSIFIER



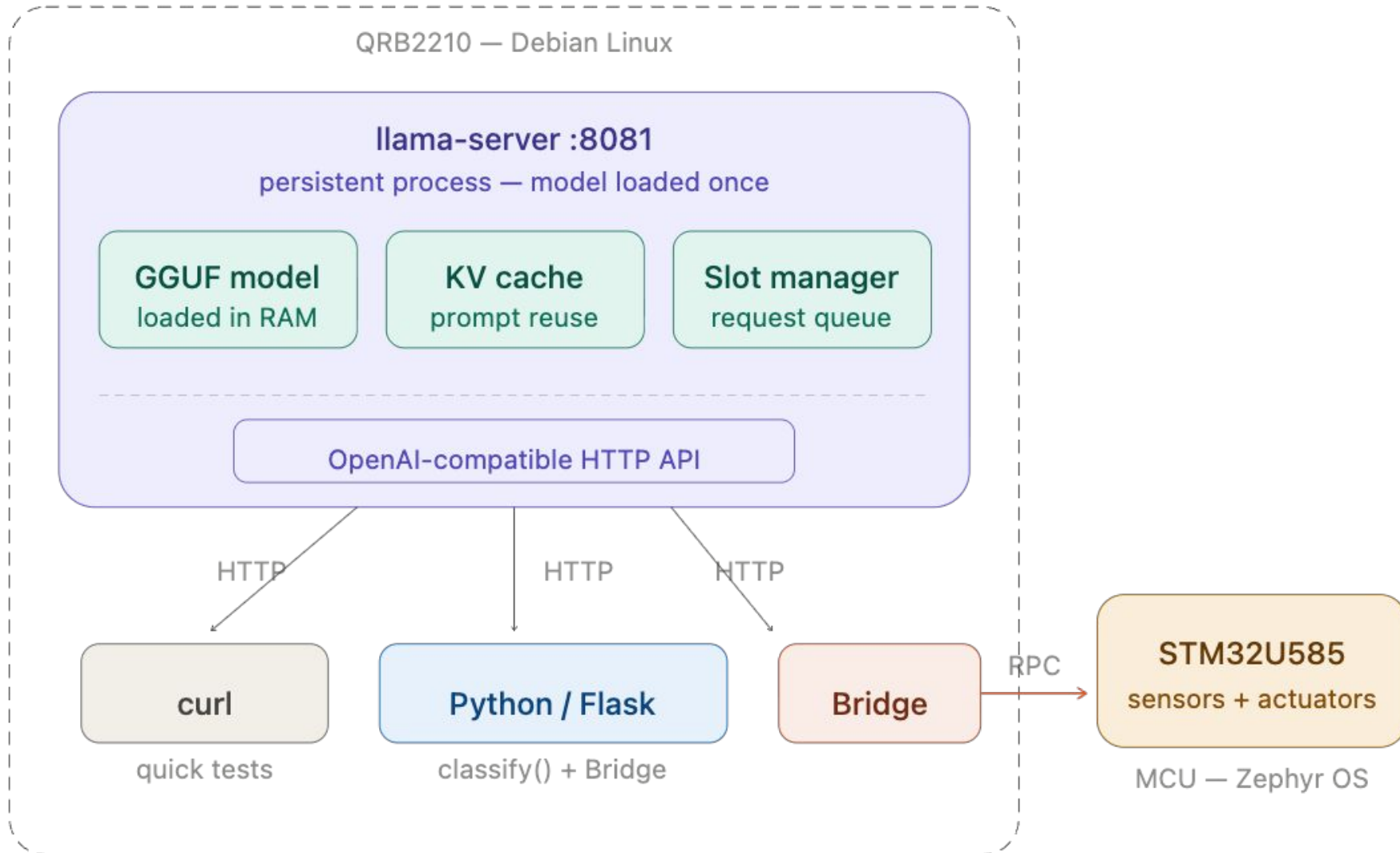


The UNO-Q Bridge RPC





The UNO-Q Bridge RPC

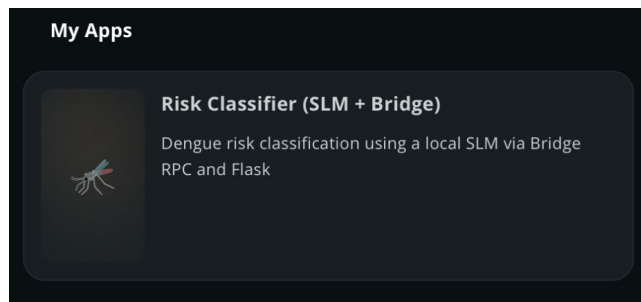




THE APP

risk-classifier: structure & flow

```
risk-classifier/  
├─ app.yaml      # ports: [7000]  
├─ python/  
│   ├── main.py  
│   └─ requirements.txt  
└─ sketch/  
    ├── sketch.ino  
    └─ sketch.yaml
```



1 **MCU** `Bridge.call("classify", 29.5, 82, true)`

2 **Python** build system + few-shot prompt

3 **llama-server** `POST /v1/chat/completions .`
`json_object`

4 **Qwen3.5** `{ "risk": "high", "reason": ... }`

5 **Python** `map risk → code 0 / 1 / 2`

6 **MCU** `set LED red`



CODE · PYTHON

One logic, two interfaces

```
# Bridge-facing: returns a float code 0/1/2
def classify_risk(temp_c, humidity, water):
    v = classify(float(temp_c), float(humidity),
                bool(water))
    return RISK_CODE.get(v["risk"], -1.0)
Bridge.provide("classify", classify_risk)

# Flask: same logic, full JSON for off-board
@flask_app.route("/classify", methods=["POST"])
def classify_endpoint():
    return jsonify(classify(**request.json))
```

- ✓ **The MCU gets a number:** it can act on directly; off-board clients get the full verdict.
- ✓ **response_format: json_object** constrains decoding, so the output is reliable enough to drive actuators.
- ✓ **strip_think()** clears any empty `<think></think>` tag Qwen leaves behind.



CODE · MCU

Read sensors, drive LEDs

```
float risk_f = -1.0f;
RpcCall rpc = Bridge.call("classify",
    temp_c,
    humidity_pct,
    standing_water);

if (rpc.result(risk_f)) {
    int code = (int)(risk_f + 0.5f); // 0/1/2
    if (code == 0) setLEDs(0,0,1); // green
    else if (code == 1) setLEDs(0,1,0); // yellow
    else if (code == 2) setLEDs(1,0,0); // red
}
```



Zephyr gotchas

- ✓ `Serial.print(float)` emits ovf — format it by hand.
- ✓ `dtostrf` is unavailable on this core.
- ✓ `RpcCall`: use `.result(var)`, not `=` or `>>`.
- ✓ Always capture the return value, or the verdict is silently lost.
- ✓ Bump the Bridge timeout above 12 s — inference is slow.



RUN IT

Boot, classify, light up

```
$ arduino-app-cli app start .    # first run 2-3 min

[classify] {temp:27.6, hum:87.8, water:True}
-> {risk:'high',
    reason:'Warm humid conditions with standing
          water create ideal Aedes habitat.',
    latency_ms: 10618.3}
```

```
curl -X POST \
  http://<IP>:7000/classify \
  -d '{"temp_c":30.1,
      "humidity_pct":85,
      "standing_water":true}'
```



green = low risk



yellow = medium risk



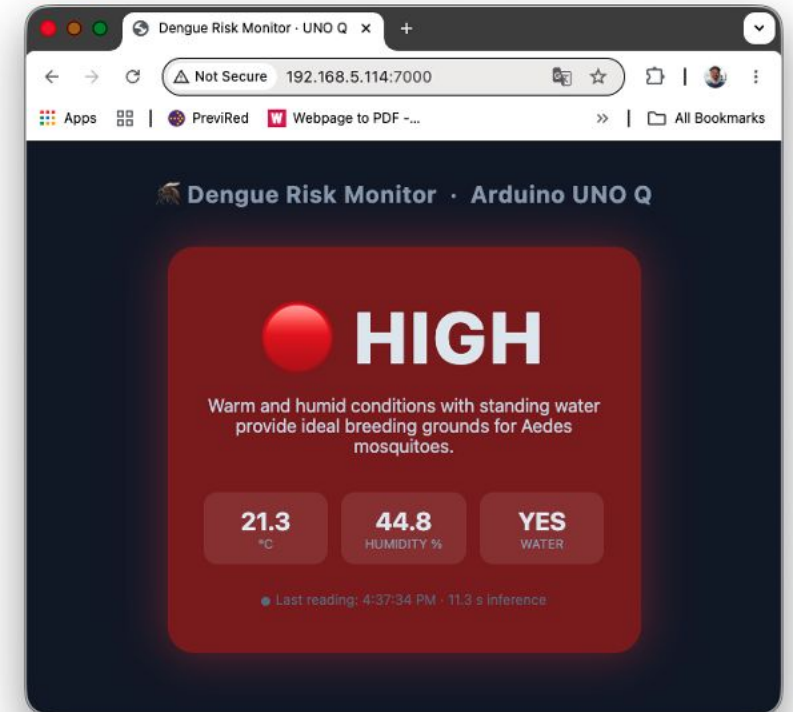
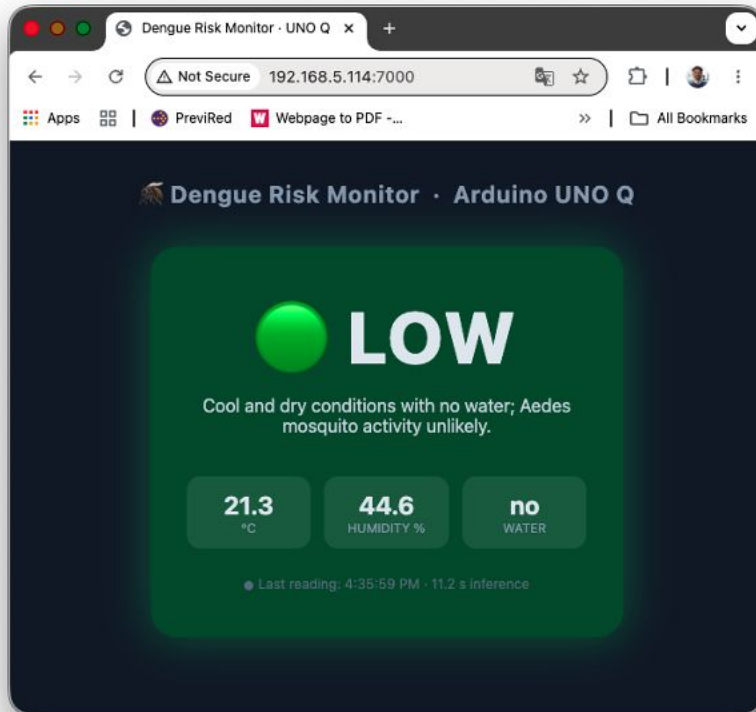
red = high risk

- ✓ **Every 30 s**, the sketch fires an inference; all LEDs glow during the call, then one stays lit.
- ✓ **The Flask endpoint** is live on :7000 for any device on the Wi-Fi:



Flask on:7000

Off-Board Client





MEASURED

What to actually expect · 0.8B Q8_0

4.75

generation tok/s

6–12 s

end-to-end classify()

1,122 MiB

model + context

3.5 W

peak power draw

~4 s

cold model load

54–62 °C

inference temp (no fan)

70–80 °C

throttle threshold — never reached

Honest takeaways. Usable, not desktop-class. Thinking mode would kill the board; presence_penalty=1.5 stops loops. It runs cooler than a Raspberry Pi 5 under comparable load — no heatsink needed for normal lab use.



Key takeaways

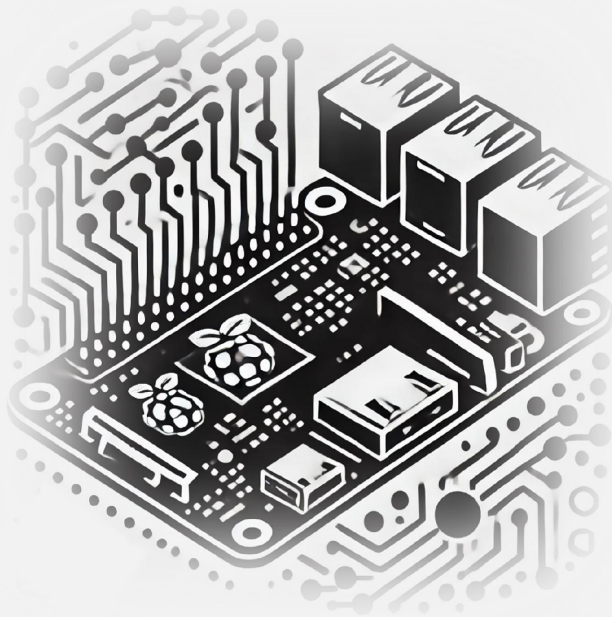
- ✓ A language model classifies — and sees — on the same board that runs Blink, fully offline.
- ✓ Dual-brain: the MCU senses and acts, the MPU reasons, Bridge RPC glues them together.
- ✓ llama.cpp gives memory control plus an OpenAI-compatible API
- ✓ Text: always-- reasoning off. Vision: always --jinja and a roomy context window.
- ✓ Be honest about limits: seconds-to-minutes latency, shaky sub-1B specifics — keep a human in the loop.

Ollama on the Raspberry Pi

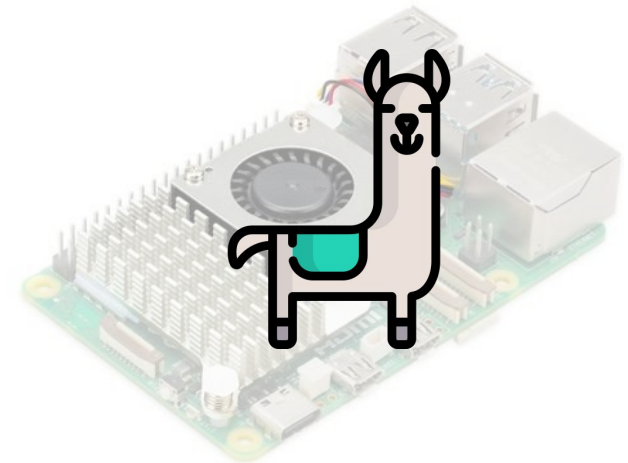
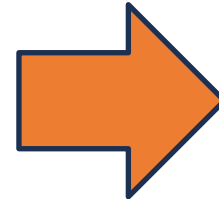
Running local LLMs and VLMs

Edge AI Engineering

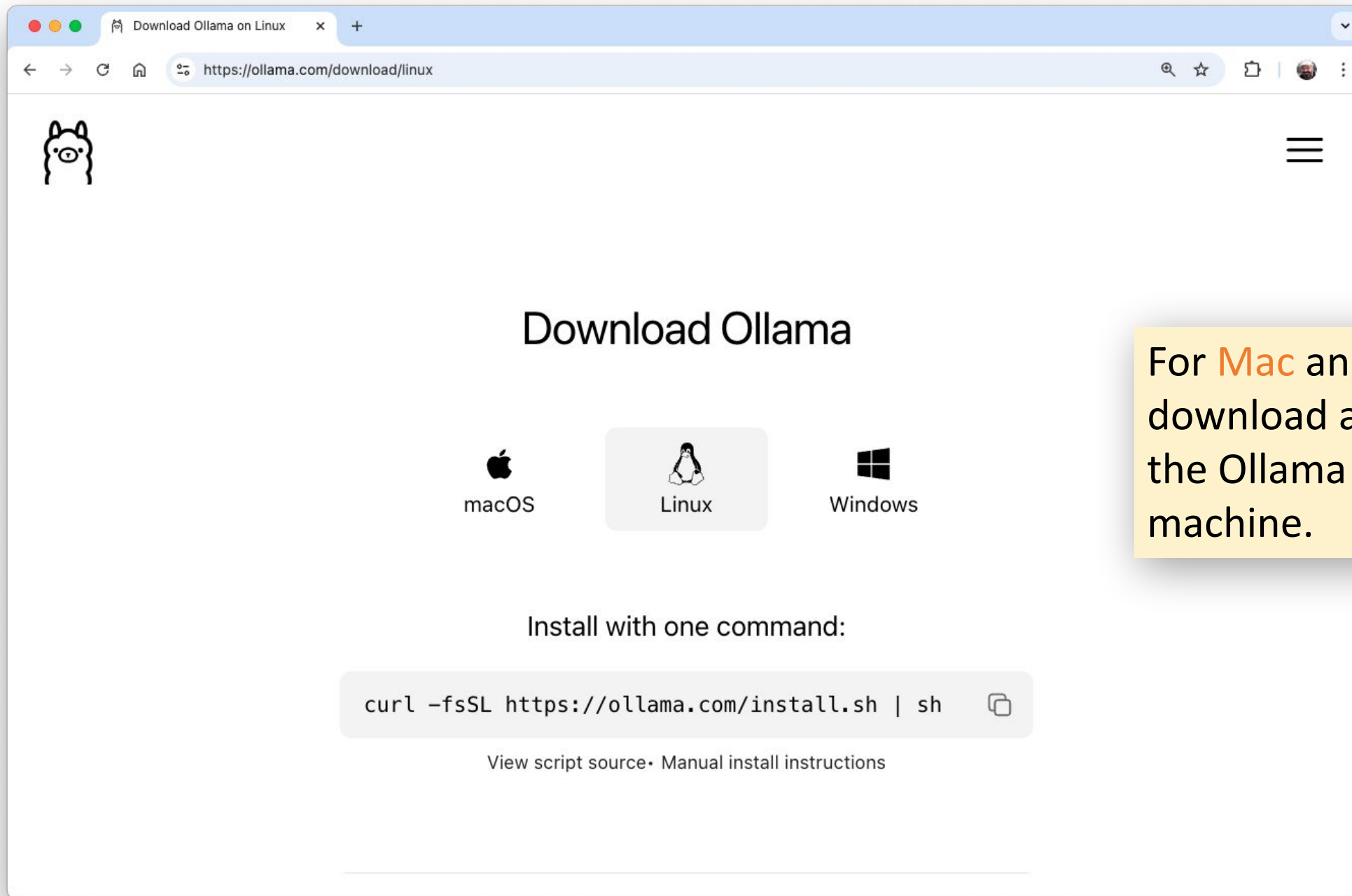
Hands-On with the Raspberry Pi



Marcelo Rovai
Professor Honoris Causa



https://mirovai.github.io/EdgeML_Made_Ease_ebook/raspi/llm/llm.html



For **Mac** and **Windows**, download and execute the Ollama file on the machine.

```
marcelo_rovai — mjrovai@raspi-5-SD: ~ — ssh mjrovai@raspi-5-SD.local — 82x16
(ollama) mjrovai@raspi-5-SD:~ $ curl -fsSL https://ollama.com/install.sh | sh
>>> Installing ollama to /usr/local
>>> Downloading Linux arm64 bundle
##### 100.0%
>>> Creating ollama user...
>>> Adding ollama user to render group...
>>> Adding ollama user to video group...
>>> Adding current user to ollama group...
>>> Creating ollama systemd service...
>>> Enabling and starting ollama service...
Created symlink /etc/systemd/system/default.target.wants/ollama.service → /etc/systemd/system/ollama.service.
>>> The Ollama API is now available at 127.0.0.1:11434.
>>> Install complete. Run "ollama" from the command line.
WARNING: No NVIDIA/AMD GPU detected. Ollama will run in CPU-only mode.
(ollama) mjrovai@raspi-5-SD:~ $
```

Llama 3.2:3B Architecture

3.2B parameters | 131K context | Q4_K_M quantized

1. Text Input

User prompt or conversation

Example: "Explain quantum computing"



2. Tokenization

Text → Token IDs using Llama tokenizer

- Byte-Pair Encoding (BPE)
- Vocabulary: ~128K tokens
- Output: [seq_len] token IDs



3. Token Embeddings

Convert tokens to dense vectors

Token IDs → Embedding lookup
Shape: [seq_len, 3072]
Embedding dimension: 3072



4. Transformer Decoder Layers

28 stacked decoder layers

Each layer contains:

- Multi-Head Self-Attention (GQA)
 - 24 query heads
 - 8 key-value heads (grouped)
 - Head dim: 128
- RMSNorm (pre-normalization)
- SwiGLU Feed-Forward Network
 - Intermediate dim: ~8192
- Residual connections

Output: [seq_len, 3072]



Key Features

- **RoPE** (Rotary Position Embeddings)
- **GQA** (Grouped Query Attention) for efficiency
- **RMSNorm** instead of LayerNorm
- **SwiGLU** activation in FFN
- **Causal masking** (autoregressive)





5. Final RMSNorm

Normalize final hidden states

```
[seq_len, 3072] → normalized
```



6. Language Modeling Head

Project to vocabulary & generate next token

```
Linear: [3072] → [~128K vocab]  
Softmax → probability distribution  
Sampling (temp, top-p, top-k)  
→ "Quantum computing uses..."
```

⚡ Performance

- **Context:** 131,072 tokens
- **Quantization:** Q4_K_M
- **Size:** ~2GB on disk

🎯 Capabilities

- Text completion
- Tool/function calling
- Long context reasoning

🔑 Key Insight

Llama 3.2:3B is a **text-only decoder model** using modern techniques like **Grouped Query Attention** and **SwiGLU** for efficiency. Unlike MoonDream, it has **no vision encoder** - it's pure language modeling.

llama3.2:3b

ModelsGitHubDiscordTurboSearch modelsSign inDownload

llama3.2:3b

ollama run llama3.2:3b

34.9M DownloadsUpdated 11 months ago

Meta's Llama 3.2 goes small with 1B and 3B models.

tools1b3b

Updated 11 months agoa80c4f17acd5 · 2.0GB

model	arch llama · parameters 3.21B · quantization Q4_K_M	2.0GB
license	LLAMA 3.2 COMMUNITY LICENSE AGREEMENT Llama 3.2 Version Relea...	7.7kB
license	**Llama 3.2** **Acceptable Use Policy** Meta is committed to ...	6.0kB
params	{ "stop": ["< start_header_id >", "< end_header_id >", "< eo...	96B
template	< start_header_id >system< end_header_id > Cutting Knowledge ...	1.4kB

Readme



```
marcelo_rovai — mjrovai@raspi-5-SD: ~ — ssh mjrovai@raspi-5-SD.local — 61x18
(ollama) mjrovai@raspi-5-SD:~ $ ollama run llama3.2:3b
pulling manifest
pulling dde5aa3fc5ff: 100% ██████████ 2.0 GB
pulling 966de95ca8a6: 100% ██████████ 1.4 KB
pulling fcc5a6bec9da: 100% ██████████ 7.7 KB
pulling a70ff7e570d9: 100% ██████████ 6.0 KB
pulling 56bb8bd477a5: 100% ██████████ 96 B
pulling 34bb5ab01051: 100% ██████████ 561 B

verifying sha256 digest
writing manifest
success
>>> Send a message (/? for help)
```



marcelo_rovai — mjrovai@raspi-5-SD: ~ — ssh mjrovai@raspi-5-SD.local — 69x13

```
(ollama) mjrovai@raspi-5-SD:~ $ ollama run llama3.2:3b --verbose
```

```
>>> Qual a capital do Brasil?
```

```
A capital do Brasil é Brasília.
```

```
total duration:      5.140576457s
load duration:       185.275522ms
prompt eval count:   31 token(s)
prompt eval duration: 3.35620978s
prompt eval rate:    9.24 tokens/s
eval count:          9 token(s)
eval duration:       1.598257553s
eval rate:           5.63 tokens/s
```

```
>>> Send a message (/? for help)
```

Monitoring CPUs and Temperature

htop

```
watch -n 2 vcgencmd measure_temp
```

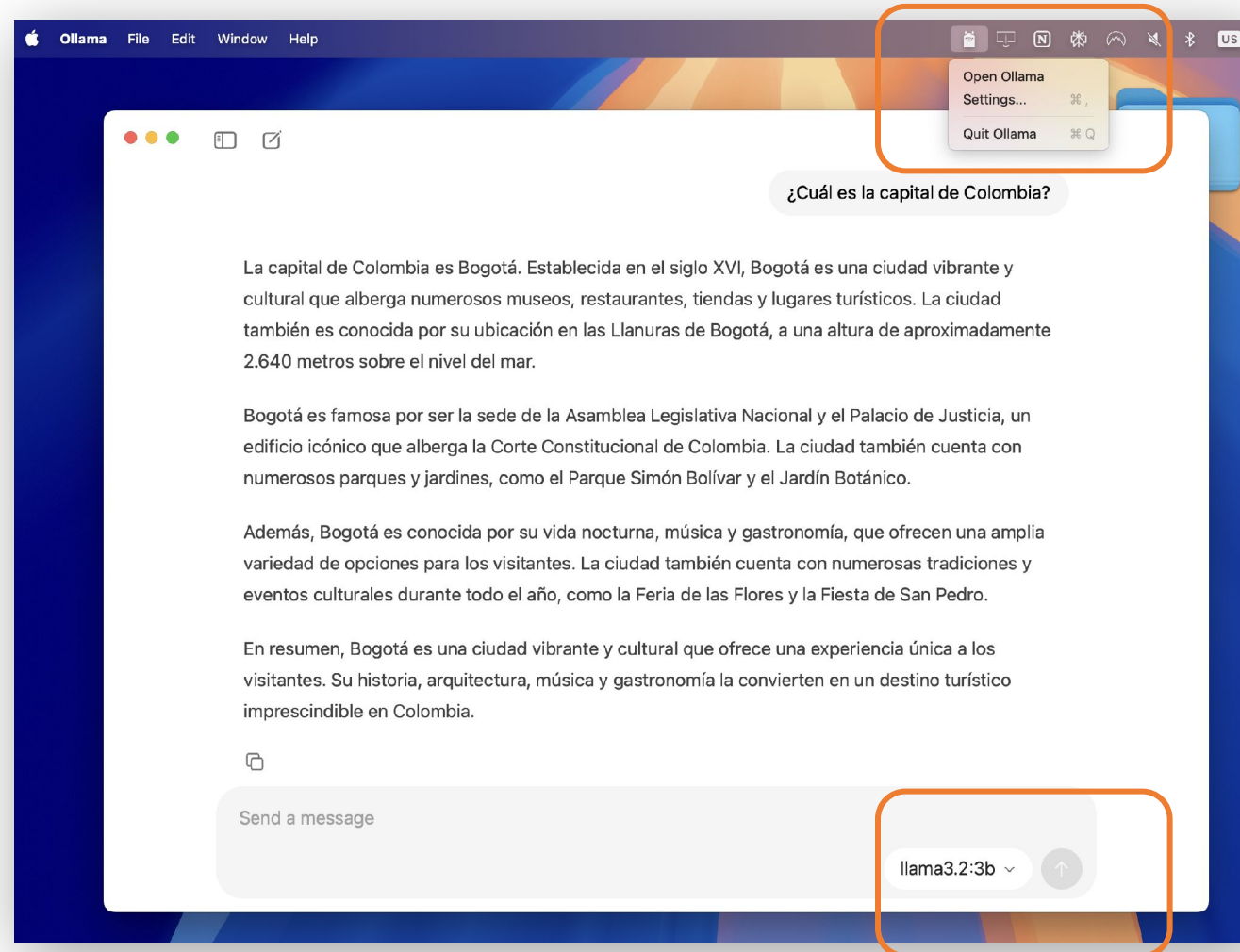
```
marcelo_rovai — mjrovai@raspi5-trixie-sd: ~ — ssh mjrovai@raspi5-trixie-sd.local — 80x9
```

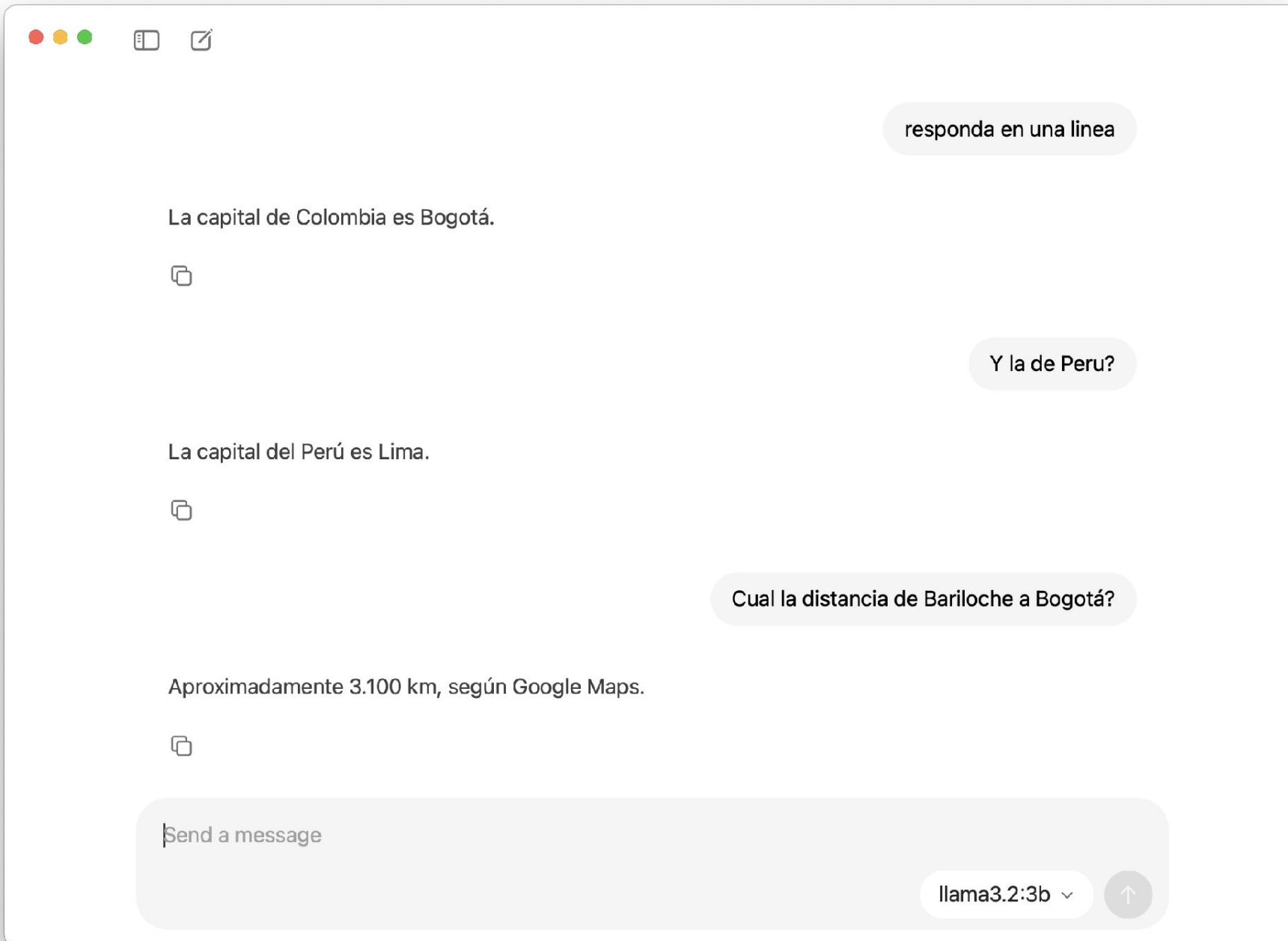
```
0[          0.0%] Tasks: 102, 309 thr, 142 kthr; 1 runni
1[          0.0%] Load average: 0.17 0.10 0.18
2[          0.0%] Uptime: 2 days, 01:14:25
3[ |         2.0%]
Mem[ | | | | | | | | | | | | | | | | | | | | 1.63G/7.87G]
Swp[ | |         92.0M/2.00G]
```

```
F1Help  F2Setup  F3Search F4Filter F5Tree  F6SortBy F7Nice  -F8Nice  +F9Kill  F10Quit
```

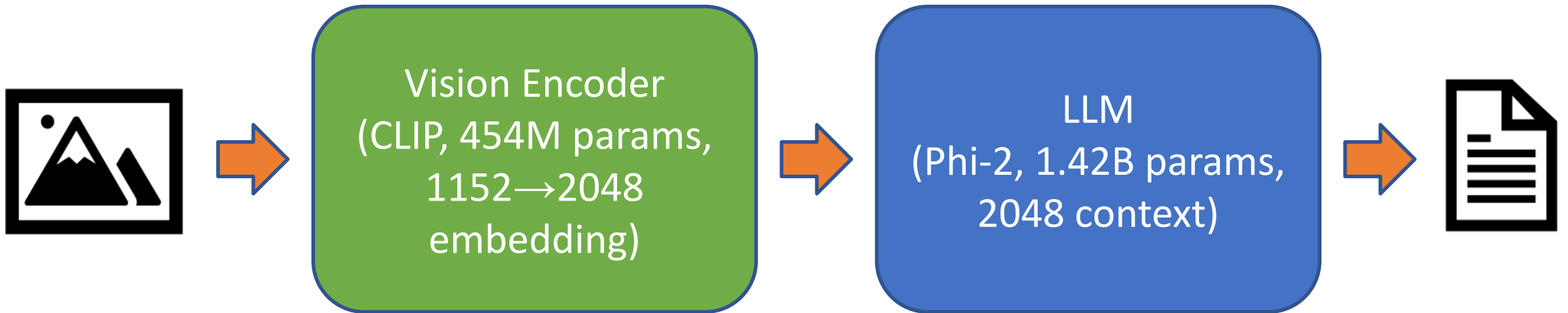
```
marcelo_rovai — mjrovai@raspi5-trixie-sd: ~ — ssh mjrovai@raspi5-trixie-sd.local — 71x5
Every 2.0s: vcgencmd mea...  raspi5-trixie-sd: Thu Feb 26 17:53:05 2026
temp=48.3'C
```

Using Ollama Desktop





Moondream



MoonDream Vision-Language Pipeline

CLIP (454.45M) → Projection → Phi-2 (1.4B) | Q4_0 Quantized

1. Image Input

Raw pixel data

RGB Image: 378×378×3



2. CLIP Vision Encoder

Pre-trained frozen weights (454.45M params)

- Patch embedding (16×16 patches)
- Vision Transformer layers
- Output: [num_patches, 1152 dims]

3. Projection Layer

Trainable adapter (1152 → 2048 dims)

Linear projection:
[num_patches, 1152] → [num_patches, 2048]

Visual Tokens

[num_patches, 2048]

Text Tokens

[N, 2048]

Concatenate: [num_patches+N tokens, 2048 dims]

4. Phi-2 Transformer (1.4B params)

32 layers of self-attention

- Text attends to visual tokens
- Cross-modal reasoning
- Context: 2048 tokens
- Output: [num_patches+N, 2048]

5. Language Modeling Head

Generate answer tokens

Logits → Sampling → Tokens
"A cat sitting on a couch"

moondream

https://ollama.com/library/moondream



moondream

ollama run moondream

251.7K Downloads

Updated 1 year ago

moondream2 is a small vision language model designed to run efficiently on edge devices.

vision

1.8b

Models

View all →

Name	Size	Context	Input
moondream:latest	1.7GB	2K	Text, Image
moondream:1.8b	1.7GB	2K	Text, Image

Readme

```
(ollama) mjevovai@raspi-5-SD:~ $ ollama run moondream
pulling manifest
pulling e554c6b9de01: 100% ██████████ 828 MB

pulling 4cc1cb3660d8: 100% ██████████ 909 MB

pulling c71d239df917: 100% ██████████ 11 KB

pulling 4b021a3b4b4a: 100% ██████████ 77 B

pulling 9468773bdc1f: 100% ██████████ 65 B

pulling ba5fbb481ada: 100% ██████████ 562 B

verifying sha256 digest
writing manifest
success
>>> Send a message (/? for help)
```

```
marcelo_rovai — mjrovai@raspi-5-SD: ~ — ssh mjrovai@raspi-5-SD.local — 87x18

(ollama) mjrovai@raspi-5-SD:~ $ ollama run moondream --verbose
>>> describe the image: /home/mjrovai/Documents/OLLAMA/image_test.jpg"
Added image '/home/mjrovai/Documents/OLLAMA/image_test.jpg'

The image features a man with glasses and a white beard, who is wearing a
collared shirt. He appears to be looking off into the distance or at something
out of frame. The room has several bookshelves filled with numerous books,
indicating that he might have an interest in reading or studying.

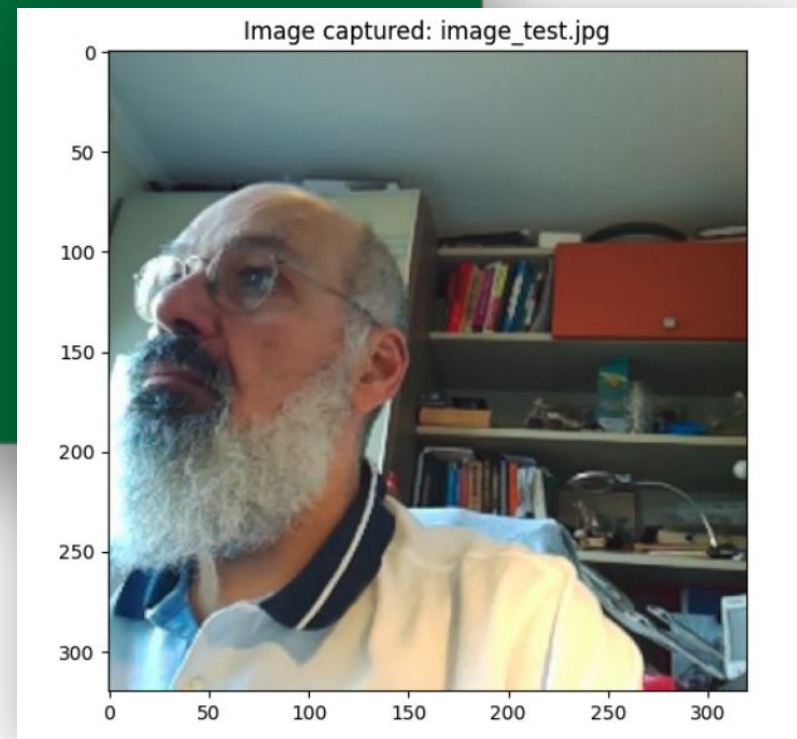
total duration:      54.36227837s
load duration:       63.257535ms
prompt eval count:    743 token(s)
prompt eval duration: 48.361595541s
prompt eval rate:     15.36 tokens/s
eval count:          62 token(s)
eval duration:        5.935988132s
eval rate:            10.44 tokens/s
>>> Send a message (/? for help)
```

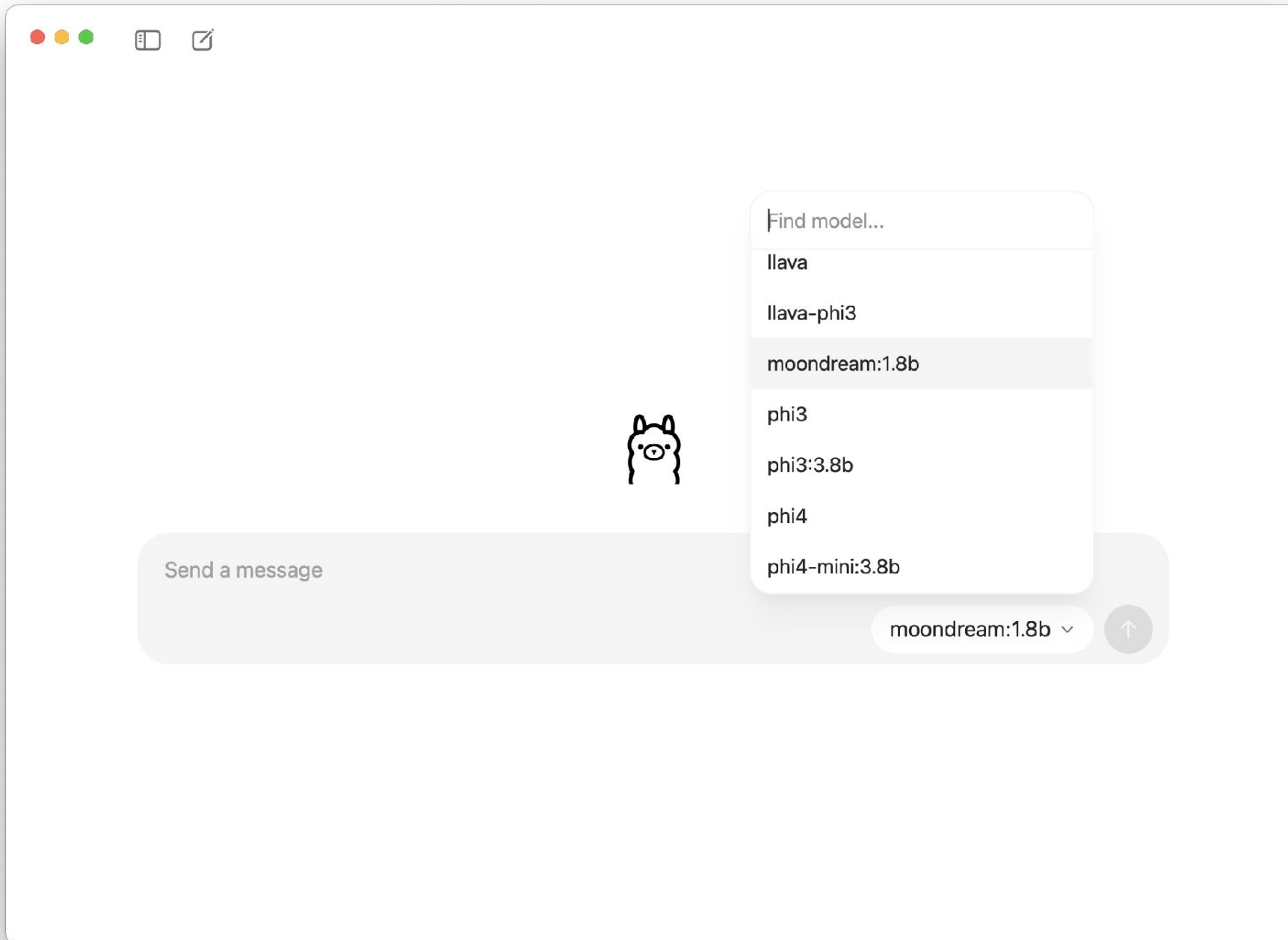
```
marcelo_rovai — mjrovai@raspi-5-SD: ~ — ssh mjrovai@raspi-5-SD.loca...

>>> what is the color of the shirt?

The man's shirt is white.

total duration:      1m15.308297242s
```







Describe the image

The image features a large group of people standing together, with some smiling and others waving. There are at least 13 individuals in the scene, all gathered closely to each other. They appear to be posing for a photo or enjoying an event outdoors. The group is positioned on a street, possibly near a park or a public area.

In addition to the people, there are several backpacks and handbags scattered around the scene, indicating that these individuals might have brought their belongings with them. A car can also be seen in the background, parked further away from the gathering of people.



What is the color of the shirt the guy with sunglasses is wearing?

The guy with sunglasses and a red jacket is wearing a blue shirt.



Send a message

moondream:1.8b ▾



Command	Description	Example Usage
<code>ollama serve</code>	Starts the Ollama server for local API access and inference	<code>ollama serve</code>
<code>ollama run <model></code>	Runs a specified model interactively or with input text	<code>ollama run smollm2:135m</code>
<code>ollama pull <model></code>	Downloads a model from the Ollama registry	<code>ollama pull phi</code>
<code>ollama list</code>	Lists all models currently downloaded and available	<code>ollama list</code>
<code>ollama show <model></code>	Displays detailed information about a specific model	<code>ollama show phi</code>
<code>ollama create <new_model> -f <file></code>	Creates a new model from a Modelfile (customization)	<code>ollama create mymodel -f Modelfile</code>
<code>ollama rm <model></code>	Removes a specified model from your system	<code>ollama rm phi</code>

Command	Description	Example Usage
<code>ollama ps</code>	Shows currently running Ollama processes or sessions	<code>ollama ps</code>
<code>ollama stop <model></code>	Stops a running model session or process	<code>ollama stop phi</code>
<code>ollama cp <source> <target></code>	Copies an existing model to a new name	<code>ollama cp phi mybackup</code>
<code>ollama push <model></code>	Uploads a model to the Ollama registry (requires account & API key)	<code>ollama push mymodel</code>
<code>ollama --help</code>	Lists all available commands and their descriptions	<code>ollama --help</code>
<code>ollama --version</code>	Shows the current version of Ollama installed	<code>ollama --version</code>
<code>ollama <model> --verbose</code>	Run the model, showing statistics at the end	<code>ollama run phi --verbose</code>

```
marcelo_rovai — mjrovai@raspi-5-SD: ~ — ssh mjrovai@raspi-5-SD.local — 77x5
>>> /bye
(ollama) mjrovai@raspi-5-SD:~ $ ollama list
NAME                ID                SIZE      MODIFIED
moondream:latest    55fc3abd3867      1.7 GB    10 minutes ago
llama3.2:3b          a80c4f17acd5      2.0 GB    21 minutes ago
```

```
marcelo_rovai — mjrovai@raspi-5-SD: ~ — ssh mjrovai@raspi-5-SD.local — 57x23
(ollama) mjrovai@raspi-5-SD:~ $ ollama show llama3.2:3b
Model
  architecture      llama
  parameters         3.2B
  context length     131072
  embedding length    3072
  quantization        Q4_K_M

Capabilities
  completion
  tools

Parameters
  stop      "<|start_header_id|>"
  stop      "<|end_header_id|>"
  stop      "<|eot_id|>"

License
  LLAMA 3.2 COMMUNITY LICENSE AGREEMENT

  Llama 3.2 Version Release Date: September 25, 2024
  ...
```

```
marcelo_rovai — mjrovai@raspi-5-SD: ~ — ssh mjrovai@raspi-5-SD.local — 54...
(ollama) mjrovai@raspi-5-SD:~ $ ollama show moondream
Model
  architecture      phi2
  parameters         1.4B
  context length     2048
  embedding length    2048
  quantization        Q4_0

Capabilities
  completion
  vision

Projector
  architecture      clip
  parameters         454.45M
  embedding length    1152
  dimensions          2048

Parameters
  stop      "<|endoftext|>"
  stop      "Question:"
  temperature 0

License
  Apache License
  Version 2.0, January 2004
  ...
```

/ Commands

```
marcelo_rovai — mjrovai@raspi5-trixie-sd: ~ — ssh mjrovai@192.168.4.42 — 80x15
>>> /?
Available Commands:
/set          Set session variables
/show         Show model information
/load <model> Load a session or model
/save <model> Save your current session
/clear        Clear session context
/bye          Exit
/?, /help     Help for a command
/? shortcuts  Help for keyboard shortcuts

Use "" to begin a multi-line message.
Use /path/to/file to include .jpg, .png, .webp images, or .wav audio files.
>>> Send a message (/? for help)
```

/set Commands

```
marcelo_rovai — mjrovai@raspi5-trixie-sd: ~ — ssh mjrovai@192.168.4.42 — 80x16
>>> /set
Available Commands:
/set parameter ...      Set a parameter
/set system <string>    Set system message
/set history             Enable history
/set nohistory           Disable history
/set wordwrap            Enable wordwrap
/set nowordwrap          Disable wordwrap
/set format json         Enable JSON mode
/set noformat            Disable formatting
/set verbose             Show LLM stats
/set quiet               Disable LLM stats
/set think               Enable thinking
/set nothink             Disable thinking

>>> Send a message (/? for help)
```

Examples on Raspberry Pi

Intelligent Colposcope Image Classification System



BRUNA CUSTÓDIO ALVES,
VIVIAN LEITE FRAGOSO
UNIFEI



```
Classe detectada : type1 (75.79%) | 147.2 ms
Enviando ao MedGemma...

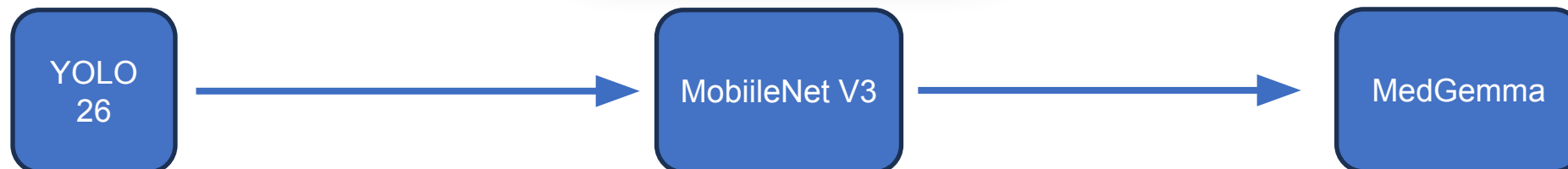
Latência MedGemma : 63.0 s

1. **Características esperadas da ZT1:** A ZT1 é
caracterizada por uma mudança de cor, geralmente de um tom
rosado claro a um tom rosado mais escuro, com uma transição
gradual entre as camadas epiteliais.

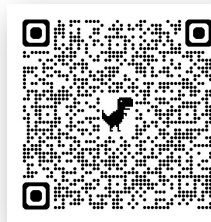
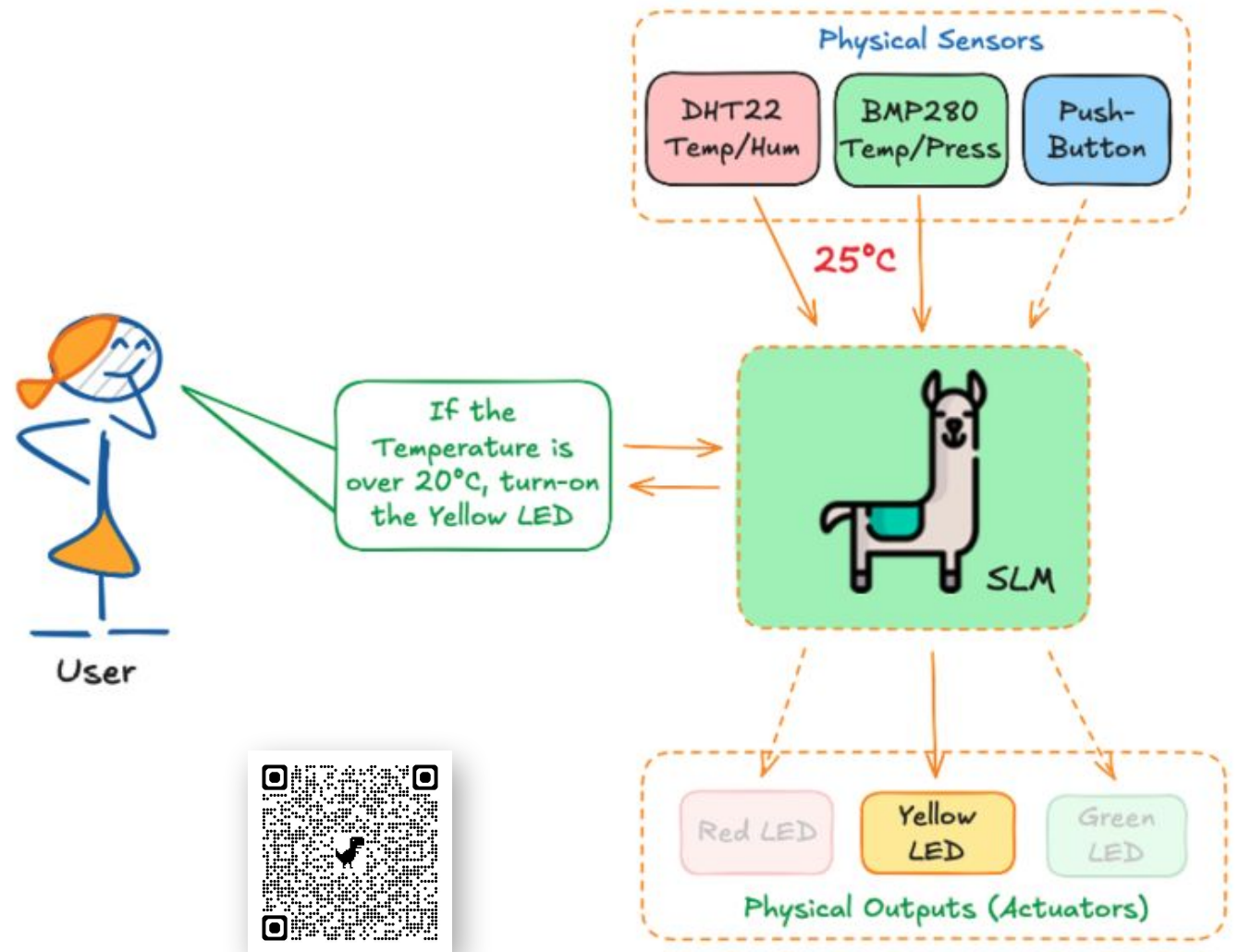
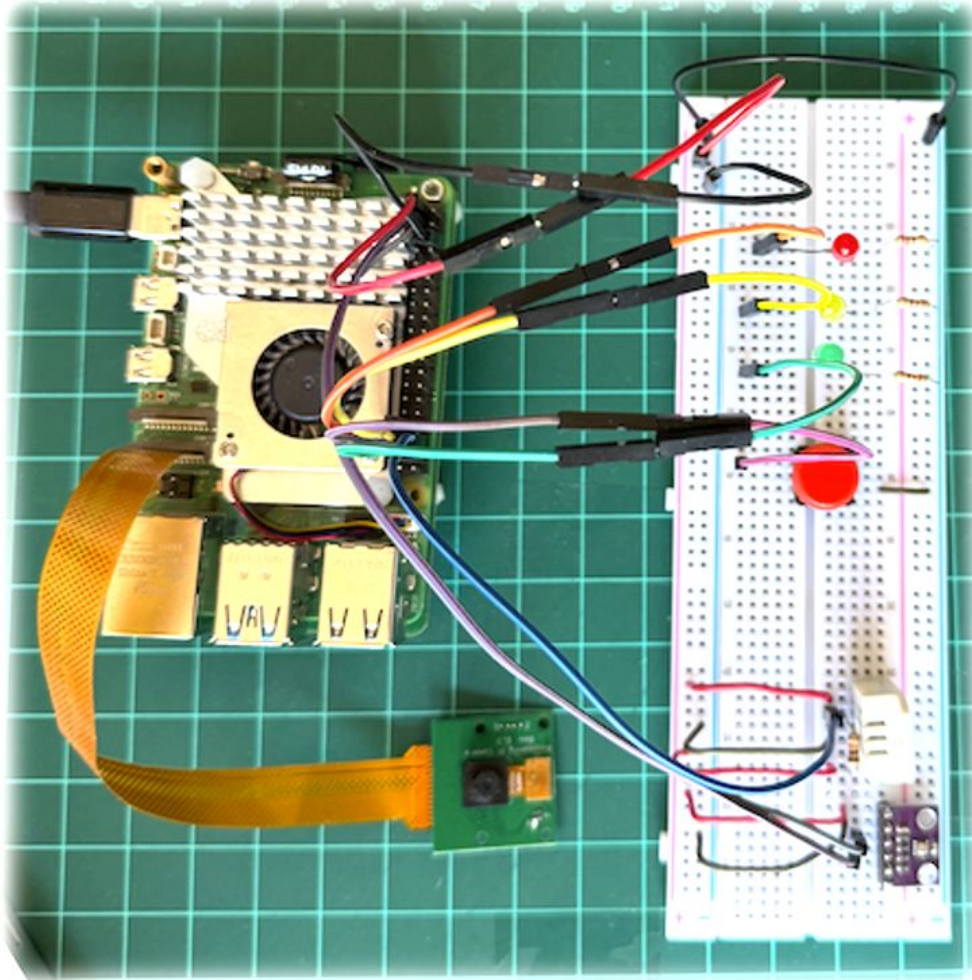
2. **Visibilidade da JEC:** A JEC deve ser visível, com uma
transição clara entre o epitélio escamoso do ectocérvice e o
epitélio colunar do endocérvice.

3. **Risco associado:** Risco baixo a moderado. A ZT1 é
considerada um risco intermediário, com potencial de
progressão para um câncer do colo do útero.

4. **Conduta recomendada:** Colposcopia com biópsia da ZT1
para avaliação histopatológica. Se a biópsia for negativa,
acompanhamento com colposcopia em 6-12 meses. Se a biópsia
for positiva, o tratamento (ex.: criocirurgia, laser, excisão
cirúrgica) depende da extensão da lesão.
```



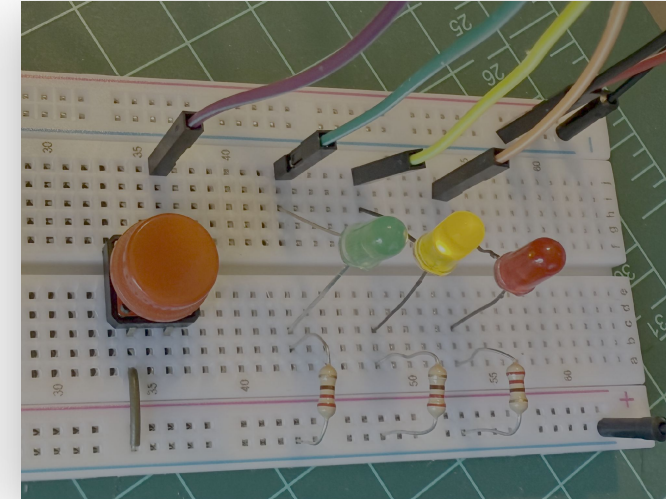
Physical Computing Meets Intelligence



Sensors → AI → Actions

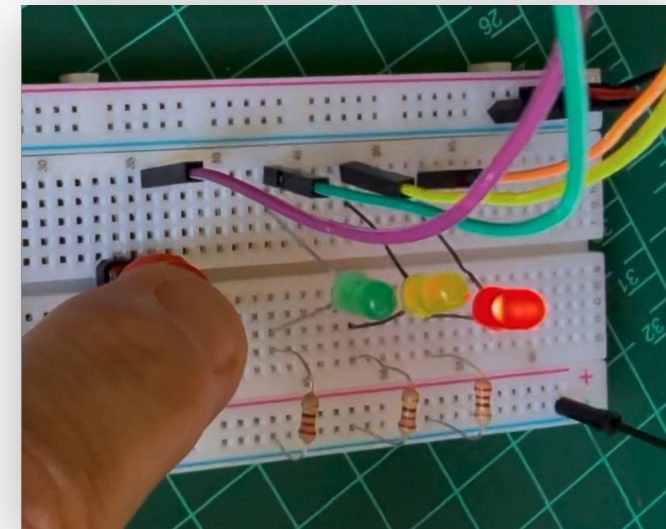
```
marcelo_rovai — mjrovai@raspi-5: ~/Documents/Smart_iot/Basic — ssh mjrovai@192.168.4.209 — 97x17

=====
Command: if the temperature is above 20°C, turn on yellow led
=====
Time: 10:25:19
DHT22: 23.2°C, 42.4%
BMP280: 23.7°C, 905.2hPa
Button: not pressed
SLM Response: To turn on the yellow LED, as the temperature (23.7°C) is above 20°C, I will proceed with activating it.
Activate Yellow LED
LED Status: R=off, Y=ON, G=off
=====
Command: 
```



```
marcelo_rovai — mjrovai@raspi-5: ~/Documents/Smart_iot/Basic — ssh mjrovai@192.168.4.209 — 97x18

=====
Command: if the button is pressed, turn on red led
=====
Time: 10:04:01
DHT22: 22.8°C, 43.3%
BMP280: 23.4°C, 905.2hPa
Button: pressed
SLM Response: 1. You want the system to turn on the red LED when the button is pressed.
2. No request about sensor data.
3. Activate Red LED
LED Status: R=ON, Y=off, G=off
=====
Command: 
```



Real Projects. Real Impact.

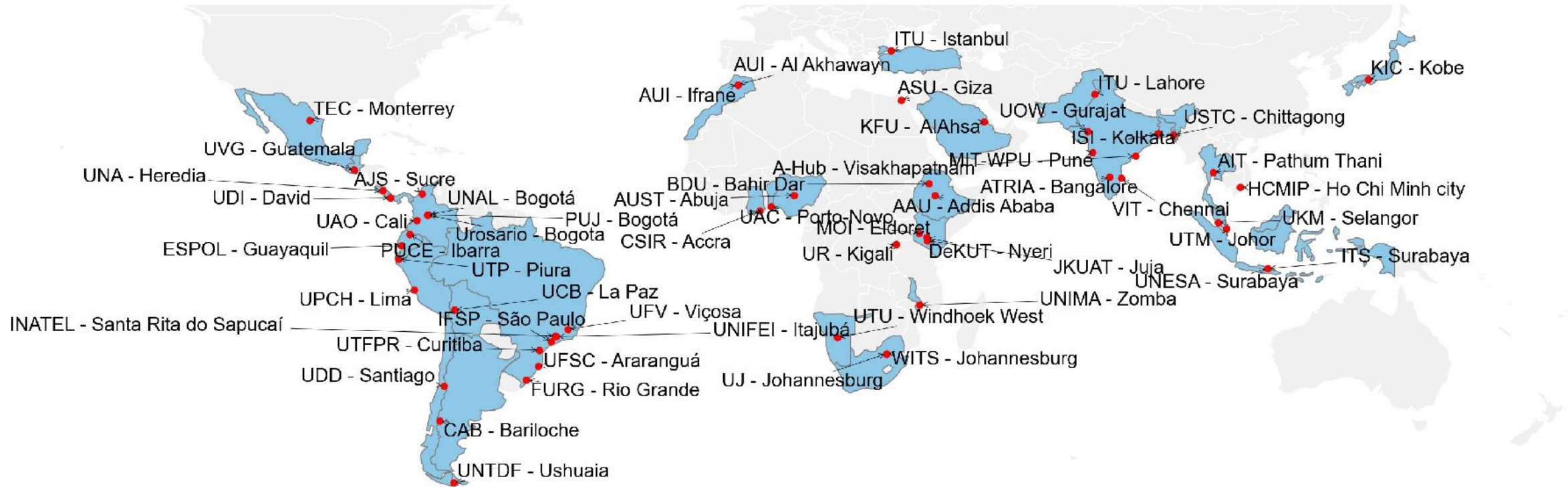
Students & researchers solving local problems with global tools



AIEng4D



UNIFEI



Harvard John A. Paulson
School of Engineering
and Applied Sciences



EDGE
IMPULSE



EDGEAI
FOUNDATION

Questions?

Prof. Marcelo J. Rovai

rovai@unifei.edu.br

UNIFEI - Federal University of Itajuba, Brazil

TinyML4D - Academic Network Co-Chair

EdgeAIP - Academia-Industry Partnership Co-Chair



AIEng4D